

SDRAM Controller ASIC Design using Verilog and OpenLane

SDRAM Controller ASIC Design using Verilog and OpenLane

Complete RTL-to-GDSII Digital ASIC Implementation

Technology: SkyWater SKY130 130 nm Open-Source PDK **Flow:** OpenLane · Yosys · OpenROAD · Magic · KLayout **Design entry:** Verilog-2001 RTL

Author <Full Name> <Email> · <GitHub profile URL> · <LinkedIn profile URL>

Institution / Organisation <Institution or Company Name> <Department>

Document Revision 1.0 · <Date>

Classification Open — portfolio and educational release

This document describes an independent engineering project. SKY130 is a trademark of SkyWater Technology Foundry. All other trademarks are the property of their respective owners.

Revision History

Rev	Date	Author	Description
0.1	<date>	<name>	Initial RTL capture and functional simulation
0.2	<date>	<name>	Synthesis closure, OpenLane flow bring-up
1.0	<date>	<name>	Full RTL-to-GDSII documentation release

Abstract

This document describes the specification, register-transfer-level design, functional verification and full physical implementation of a synchronous DRAM (SDRAM) memory controller, taken from behavioural Verilog through to a manufacturable GDSII layout in the SkyWater SKY130 130 nm open-source process using the OpenLane flow.

The controller presents a simple synchronous request/acknowledge interface to a host processor and translates single-word read transactions into the multi-cycle command sequences demanded by a JEDEC-style SDRAM device. It is partitioned into four cooperating RTL modules: an **address decoder** that maps a flat 24-bit host address onto the bank/row/column geometry of the memory array; a **bank manager** that maintains per-bank open-row state so that repeat accesses to an already-active row can bypass the row-activation penalty; an **initialization unit** that executes the mandatory power-up sequence (stabilisation delay, precharge-all, auto-refresh, mode register load) before any user traffic is admitted; and a **command generator** containing the main control finite state machine and the timing counters that enforce the device's tRCD, tRP, tRAS and CAS-latency constraints.

The address map was derived from the memory geometry and verified against simulation: `bank = address[23:22]`, `row = address[21:10]`, `column = address[9:2]`, with the two least-significant bits reserved as a byte offset within the 32-bit data word. This addresses a 4-bank × 4096-row × 256-column array.

Functional verification was performed with a directed Verilog testbench under Xilinx Vivado's simulator, exercising reset behaviour, the initialization sequence, row-miss transactions requiring a full ACTIVATE–READ–PRECHARGE sequence, and row-hit transactions. Waveform inspection confirmed correct address field extraction, correct command strobe ordering, and correct latching of the decoded address at request acceptance.

Physical implementation was carried out in OpenLane: RTL elaboration and technology mapping in Yosys against the SKY130 high-density standard-cell library, followed by floorplanning, power distribution network insertion, global and detailed placement, clock tree synthesis, global and detailed routing, parasitic extraction, static timing analysis, and signoff design-rule and layout-versus-schematic checking in Magic and KLayout. The flow completed to a signed-off GDSII database.

The project's engineering value lies less in the complexity of the controller itself than in the completeness of the path traversed: every abstraction level between a behavioural description and a mask database was exercised, and each level's failure modes — RTL width mismatches, synthesis inference surprises, congestion-driven routing failures, antenna and density rule violations — was encountered and resolved directly.

Index terms — SDRAM controller, memory controller, ASIC design, RTL-to-GDSII, OpenLane, SKY130, Yosys, OpenROAD, static timing analysis, physical design, digital VLSI.

Acknowledgement

The open-source EDA ecosystem is the reason this project was possible at all. A decade ago, a complete digital ASIC flow was accessible only to those with a commercial tool licence costing more than a car. The work of the Yosys, OpenROAD, Magic, KLayout and OpenLane maintainers, and the decision by SkyWater Technology and Google to publish a manufacturable 130 nm process design kit under a free licence, collectively removed that barrier. Anyone with a laptop and persistence can now take a design to GDSII.

Thanks are due to the maintainers and contributors of those projects, to the Efabless team for the OpenLane integration and its documentation, and to the community contributors whose issue threads and forum posts resolved several of the tool-flow problems documented in Chapter 10 — problems that would otherwise have consumed considerably more time than the design work itself.

Thanks also to <supervisor / mentor / colleague names> for technical review and guidance, and to <institution / organisation> for providing the compute resources on which the implementation runs were executed.

Table of Contents

Front Matter - Revision History - Abstract - Acknowledgement - List of Figures - List of Tables - Glossary and Abbreviations

Chapter 1 — Introduction - 1.1 Semiconductor Design as an Engineering Discipline - 1.2 What VLSI Means in Practice - 1.3 ASIC versus FPGA - 1.4 The Memory Wall - 1.5 Why SDRAM Exists - 1.6 Why a Controller Is Necessary - 1.7 Industry Context and Applications - 1.8 Scope and Structure of

Chapter 2 — Project Objectives - 2.1 Primary Objectives - 2.2 Functional Requirements - 2.3 Design Constraints - 2.4 Performance Targets - 2.5 Area and Power Considerations - 2.6 Modularity and Scalability - 2.7 Explicit Non-Goals

Chapter 3 — Background Theory - 3.1 The Memory Hierarchy - 3.2 DRAM Cell Physics and Its Consequences - 3.3 From Asynchronous DRAM to SDRAM - 3.4 Array Organisation: Banks, Rows, Columns - 3.5 The Row Buffer and Locality - 3.6 The SDRAM Command Set - 3.7 Timing Parameters - 3.8 CAS Latency and the Read Data Pipeline - 3.9 Refresh - 3.10 The Mode Register - 3.11 The Power-Up Initialization Sequence - 3.12 Burst Transfers

Chapter 4 — System Architecture - 4.1 Architectural Overview - 4.2 Top-Level Interface Definition - 4.3 Address Decoder - 4.4 Bank Manager - 4.5 Initialization Unit - 4.6 Command Generator and Control FSM - 4.7 Timing Control Logic - 4.8 Inter-Module Protocol

Chapter 5 — RTL Design Methodology - 5.1 Language and Coding Standard - 5.2 Module Hierarchy - 5.3 Synchronous Design Discipline - 5.4 Reset Strategy - 5.5 Clocking - 5.6 State Machine Coding Style - 5.7 Parameterisation - 5.8 Signal Naming Convention - 5.9 Lint and Synthesis-Awareness

Chapter 6 — Functional Verification - 6.1 Verification Strategy - 6.2 Testbench Architecture - 6.3 Stimulus Set - 6.4 Waveform Analysis - 6.5 Verification Results - 6.6 Coverage Discussion - 6.7 Known Limitations and Verification Gaps

Chapter 7 — ASIC Implementation Flow - 7.1 Flow Overview - 7.2 Synthesis - 7.3 Technology Mapping - 7.4 Floorplanning - 7.5 Power Distribution Network - 7.6 Placement - 7.7 Clock Tree Synthesis - 7.8 Routing - 7.9 Parasitic Extraction and Static Timing Analysis - 7.10 Physical Verification: DRC - 7.11 Physical Verification: LVS - 7.12 Antenna and Density Checks - 7.13 GDSII Stream-Out

Chapter 8 — Tools and Environment

Chapter 9 — Results

Chapter 10 — Design Challenges and Resolutions

Chapter 11 — Future Work

Chapter 12 — Conclusion

References

Appendices A–F

List of Figures

Figure	Caption
1.1	Memory hierarchy: capacity, latency and cost per bit
1.2	ASIC versus FPGA implementation trade-off space
1.3	Processor–memory performance divergence (“the memory wall”)
3.1	DRAM storage cell and destructive read mechanism
3.2	SDRAM device internal organisation: banks, rows, columns, row buffer
3.3	Row-hit and row-miss access paths through the row buffer
3.4	SDRAM command truth table and pin encoding
3.5	Read transaction timing diagram with tRCD, CAS latency and tRP annotated
3.6	Mode register bit field allocation
3.7	Power-up initialization sequence flowchart
3.8	Burst read timing for BL = 4
4.1	SDRAM controller module architecture
4.2	Generic SDRAM controller reference architecture
4.3	Detailed signal-level architecture
4.4	Elaborated RTL schematic showing the four-module hierarchy
4.5	Address field extraction from the 24-bit host address
4.6	Bank manager per-bank state array and update conditions
4.7	Control FSM state transition diagram
4.8	Initialization unit state transition diagram
5.1	Design hierarchy as presented by the synthesis front end
5.2	Two-always-block FSM coding template
6.1	Testbench block diagram
6.2	Full-run simulation waveform: reset, initialization and four transactions
6.3	Detail waveform: ACTIVATE–READ–PRECHARGE sequence for address 0xFEDCBA
6.4	Bank manager internal signal inspection during row-hit sequence
7.1	RTL-to-GDSII flow diagram
7.2	Synthesis hierarchy and cell mapping report
7.3	Initial floorplan with core area and I/O pin placement
7.4	Power distribution network straps and standard-cell rail connection
7.5	Global placement result and cell density heat map
7.6	Detailed placement result, legalised cell rows
7.7	Clock tree structure after CTS
7.8	Global routing congestion map
7.9	Detailed routing result, all metal layers
7.10	Metal layer utilisation by layer
8.1	Tool flow and data handoff between tools
9.1	Final GDSII layout, full die
9.2	Final GDSII layout with net labels (Magic)
9.3	Final GDSII layout, core detail (Magic)
9.5	DRC signoff report summary
9.6	LVS signoff report summary
9.7	Post-route timing histogram
9.8	Power breakdown by category
A.1	Complete module hierarchy tree
C.1	Repository folder structure

Figures shown in **bold** correspond to screenshots already captured and included in [/images](#) . The remainder are placeholders to be filled from the OpenLane run directory as described in Appendix D.

List of Tables

Table	Caption
1.1	ASIC versus FPGA: quantitative comparison
1.2	Memory technology characteristics by tier
2.1	Functional requirement matrix
2.2	Design constraint summary
3.1	SDRAM command encoding truth table
3.2	Key SDRAM timing parameters and their physical origin
3.3	Timing parameter conversion to clock cycles at target frequency
3.4	Mode register field definitions
3.5	Initialization sequence step table
4.1	Top-level port list
4.2	Address field allocation
4.3	Address decoder port list
4.4	Bank manager port list
4.5	Bank manager state update truth table
4.6	Initialization unit port list
4.7	Command generator port list
4.8	Control FSM state encoding and exit conditions
5.1	Module hierarchy and source file mapping
5.2	Signal naming convention
6.1	Directed test case list
6.2	Expected versus observed decode results
6.3	Verification coverage assessment
7.1	Flow stage inputs, outputs and tools
7.2	Synthesis configuration parameters
7.3	Floorplan configuration parameters
7.4	Placement configuration parameters
7.5	CTS configuration parameters
7.6	Routing configuration parameters
8.1	Tool inventory, version and role
9.1	Synthesis results summary
9.2	Area utilisation summary
9.3	Static timing analysis summary
9.4	Power estimation summary
9.5	Physical verification signoff summary
10.1	Issue log: symptom, root cause, resolution
11.1	Future work items with estimated effort and dependency ordering
B.1	RTL source file descriptions

Glossary and Abbreviations

Term	Definition
ASIC	Application-Specific Integrated Circuit
BEOL	Back End Of Line — the metal interconnect layers
CAS	Column Address Strobe
CL	CAS Latency — cycles from READ command to first valid data
CTS	Clock Tree Synthesis
DEF	Design Exchange Format — physical layout database
DRC	Design Rule Check
FEOL	Front End Of Line — the transistor layers
GDSII	Graphic Data System II — the mask layout stream format
LEF	Library Exchange Format — abstract cell physical view
LVS	Layout Versus Schematic
PDK	Process Design Kit
PDN	Power Distribution Network
RAS	Row Address Strobe
RTL	Register Transfer Level
SDC	Synopsys Design Constraints
SDRAM	Synchronous Dynamic Random Access Memory
SPEF	Standard Parasitic Exchange Format
STA	Static Timing Analysis
tRAS	Row active time — minimum ACTIVATE to PRECHARGE interval
tRCD	Row-to-Column Delay — ACTIVATE to READ/WRITE interval
tRP	Row Precharge time — PRECHARGE to next ACTIVATE interval
WNS	Worst Negative Slack
TNS	Total Negative Slack

Chapter 1 — Introduction

1.1 Semiconductor Design as an Engineering Discipline

Integrated circuit design is unusual among engineering disciplines in the sheer distance between the level at which a human expresses intent and the level at which the artefact physically exists. A designer writes a few hundred lines describing behaviour in terms of registers and logical operations. What is eventually manufactured is a set of photolithographic masks describing polygons a few hundred nanometres across, patterning several hundred million transistors. No individual reasons about both ends of that gap simultaneously. The discipline exists as a stack of abstraction layers, each with its own representation, its own tooling, and its own class of failure.

The stack has a canonical shape:

Specification	"what must it do"
Behavioural / RTL	Verilog, SystemVerilog, VHDL
Gate-level netlist	AND, OR, flip-flops from a standard-cell library
Placed netlist	every cell assigned an (x, y) coordinate
Routed netlist	every net realised as metal wires and vias
Mask layout (GDSII)	polygons per process layer
Silicon	

Each downward transition is performed by a synthesis or optimisation tool, and each is verified by an independent check that the transition preserved meaning. Synthesis is checked by equivalence checking or simulation; placement and routing are checked by static timing analysis; the final layout is checked against the schematic by LVS and against the foundry's manufacturability rules by DRC. The discipline is therefore as much about *verification of transformations* as about design.

This project's purpose was to traverse the entire stack once, on a design small enough to be understood completely, so that each abstraction layer and each verification step became concrete rather than theoretical.

1.2 What VLSI Means in Practice

"Very Large Scale Integration" is a historical term — it originally denoted devices with more than roughly ten thousand transistors, a threshold crossed decades ago. In current usage it simply denotes the discipline of digital integrated circuit design. What actually distinguishes it from other digital design work is the set of constraints that become binding at scale:

- **Timing is a first-class design object.** In board-level design, propagation delay is usually a footnote. On-chip, at a 100 MHz clock, a signal has 10 ns to traverse combinational logic, wire capacitance, and setup margin. Whether a design works is determined by static timing analysis, not by simulation.
- **Wires cost more than gates.** Beyond roughly the 250 nm node, interconnect delay came to dominate gate delay. Placement determines wire length; wire length determines delay and dynamic power. Physical design is therefore not a mechanical back-end activity — it materially determines whether an architecture is viable.
- **Errors are permanent.** A mask set is expensive and a fabrication cycle takes weeks to months. There is no patch. This asymmetry is why the verification effort on a commercial chip routinely exceeds the design effort by a factor of two or more.
- **Power is a hard constraint.** Both dynamic switching power and static leakage scale with device count. Thermal limits, not transistor availability, bound modern designs.

1.3 ASIC versus FPGA

The two implementation targets for digital logic differ fundamentally. An FPGA is a manufactured device containing a fixed fabric of lookup tables, flip-flops, block RAMs, DSP slices and programmable interconnect; a design is realised by loading a configuration bitstream that programs that fabric. An ASIC is manufactured as the design: standard cells are placed and wired specifically for it.

Table 1.1 — ASIC versus FPGA: quantitative comparison

Dimension	ASIC	FPGA
Non-recurring engineering cost	High — mask set, tooling, verification	Low — device purchase only
Per-unit cost at volume	Very low	High, roughly flat with volume
Break-even volume	Typically 10 ⁴ –10 ⁶ units, node-dependent	Favoured below that
Performance (fmax)	Highest — logic is dedicated	Typically 3–10× lower
Area efficiency	Highest	Poor — LUT-based logic is heavily over-provisioned
Power efficiency	10–100× better	Static power of unused fabric is always paid
Design cycle	Months to years	Hours to days
Post-fabrication change	Impossible	Trivial — reload bitstream
Design risk	High — a functional bug is a respin	Low
Analogue / custom content	Possible	Not possible
Typical use	High-volume products, extreme power/performance targets	Prototyping, low volume, evolving standards

Figure 1.2 placeholder — [Insert Figure 1.2: ASIC vs FPGA trade-off space] Suggested rendering: a two-axis plot, volume on x, total cost of ownership on y, with the two cost curves crossing at the break-even volume.

The two are not rivals so much as consecutive stages. Commercial memory-controller IP is routinely validated on FPGA before ASIC implementation, because an FPGA build takes an afternoon and a mask set does not. This project used both targets in exactly that relationship: an FPGA toolchain for RTL elaboration and simulation, and the OpenLane ASIC flow for the physical implementation that is the subject of this report.

1.4 The Memory Wall

Processor performance and DRAM latency have diverged for four decades. Logic speed improved roughly 50–60 % per year through the 1990s while DRAM access latency improved roughly 7 % per year, because DRAM latency is bounded by the physics of charging a bit line through a small storage capacitor rather than by transistor switching speed. The accumulated gap — the “memory wall” — means that a modern core stalled on a main-memory access wastes hundreds of cycles.

Figure 1.3 placeholder — [Insert Figure 1.3: Processor–memory performance divergence] Log-scale y axis, years on x, two curves diverging from a common origin around 1980.

Two structural responses followed, and both are relevant here. The first is the memory hierarchy — inserting progressively smaller and faster caches between the core and DRAM. The second is to make DRAM accesses *efficient in aggregate* even though each is slow: pipelining, banking, bursting, and scheduling. The second response is the memory controller’s job, and it is the reason this component is architecturally significant out of proportion to its gate count.

Table 1.2 — Memory technology characteristics by tier

Tier	Technology	Typical capacity	Typical latency	Volatile	Cost per bit
Register file	Flip-flops / custom cells	100 s of bytes	< 1 cycle	Yes	Highest
L1 cache	6T SRAM	32–64 KB	3–5 cycles	Yes	Very high
L2 / L3 cache	6T SRAM	256 KB – 64 MB	12–50 cycles	Yes	High
Main memory	1T1C DRAM (SDRAM/DDR _x)	1–512 GB	60–120 ns	Yes	Low
Storage	NAND flash	128 GB – 8 TB	20–100 μs	No	Very low

Figure 1.1 placeholder — [Insert Figure 1.1: Memory hierarchy pyramid]

1.5 Why SDRAM Exists

A DRAM bit cell is one transistor and one capacitor. That is the entire reason DRAM dominates main memory: an SRAM cell needs six transistors, so DRAM achieves roughly four to six times the bit density at the same node, and density is what determines cost per bit. Everything difficult about DRAM follows from that same minimal cell:

- The capacitor leaks, so the contents must be periodically **refreshed**.
- The stored charge is tiny, so reading it requires **sense amplifiers**, and reading **destroys** the stored value — it must be written back.
- The sense amplifiers operate on an entire row at once, so the access granularity is a **row**, not a bit.

Early DRAM was asynchronous: the controller asserted **RAS** and **CAS** and waited a specified time. This has two costs. It requires the controller to

generate precise analogue delays, and it forbids pipelining — the device does one thing at a time.

Synchronous DRAM changed the interface, not the array. All commands and data are registered on a clock edge. This has three consequences that matter architecturally:

1. **Timing becomes countable.** The controller counts clock cycles rather than generating delays. This is what makes a controller a synthesisable digital block.
2. **The device becomes pipelined.** With a registered interface, a new command can be issued before the previous one has produced data.
3. **Banks become independently usable.** Multiple internal arrays can have rows open simultaneously, so a row activation in one bank overlaps with a data transfer in another.

1.6 Why a Controller Is Necessary

An SDRAM device does not present a memory interface. It presents a *command* interface with sequencing obligations. A processor issuing “read 32 bits from address 0xFEDCBA” cannot be connected to it. Between the two, something must:

Obligation	Consequence for the controller
Address is not flat	Decode the host address into bank, row and column fields
A row must be opened before access	Issue ACTIVATE, then wait tRCD
Only one row per bank may be open	Track open-row state per bank; PRECHARGE before switching rows
Data appears a fixed number of cycles after READ	Pipeline the data capture by CAS latency
A row must stay open a minimum time	Enforce tRAS before allowing PRECHARGE
Precharge takes time	Enforce tRP before the next ACTIVATE
Charge leaks away	Issue AUTO REFRESH at the required average interval
The device powers up in an undefined state	Execute the mandated initialization sequence before any traffic

Every one of these obligations maps to a specific piece of hardware in the design described in Chapter 4. That mapping — obligation to module — is the architectural core of this report.

There is a second, less obvious reason controllers matter: they are where memory *system* performance is decided. Two controllers meeting identical timing specifications can differ by a factor of two or more in delivered bandwidth, depending on how they exploit row locality and bank parallelism. A controller that closes the row after every access pays a full ACTIVATE–READ–PRECHARGE penalty on every transaction. One that leaves the row open and detects a repeat access to it pays only the READ. Modern controllers extend this with request reordering, write batching, and refresh scheduling. This project implements the first and most important of these optimisations — open-row tracking — and identifies the remainder as future work in Chapter 11.

1.7 Industry Context and Applications

Device manufacturers. The DRAM market is a three-supplier oligopoly: Samsung, SK Hynix and Micron account for the overwhelming majority of global output. These companies design both the memory devices and, increasingly, the controller and interface IP around them. Micron’s business in particular spans commodity DDR, LPDDR for mobile, GDDR for graphics, and HBM for accelerators — each of which is the same array technology behind a different interface and controller architecture.

Controller and PHY IP vendors. Very few SoC teams design their own DDR controller from scratch. Synopsys (DesignWare), Cadence (Denali), and Rambus license controller and PHY subsystems that represent hundreds of engineer-years of work, largely because the analogue PHY — training, levelling, per-bit deskew — is extremely difficult. The digital controller portion, which is what this project implements, is the more approachable half.

Application domains and what each demands:

Domain	Memory technology	Dominant controller requirement
Servers / data centre	DDR4, DDR5, RDIMM	Capacity, ECC, RAS features, many-channel scheduling
Mobile / handset	LPDDR4X, LPDDR5	Energy per bit; aggressive power-down and self-refresh
Embedded / industrial	SDR SDRAM, DDR3, LPDDR2	Cost, simplicity, determinism, wide temperature range
Automotive	LPDDR4, DDR3 with ECC	Functional safety (ISO 26262), ECC, temperature, longevity
Graphics	GDDR6, GDDR6X	Raw bandwidth; deep bank parallelism
AI accelerators	HBM2E, HBM3, HBM4	Extreme bandwidth via 1024-bit-wide stacked interfaces
Networking	DDR4, HBM	Deterministic latency, packet-buffer access patterns
FPGA / prototyping	DDR3, DDR4	Soft controller flexibility

Single-data-rate SDRAM, as targeted here, is no longer used in performance-oriented systems. It remains in production for cost-sensitive embedded applications, and it remains the correct pedagogical starting point: every architectural concept in a DDR5 controller — bank state tracking, timing

enforcement, refresh management, command scheduling — is present in an SDR controller, without the additional burden of source-synchronous data strobes, write levelling, read training and on-die termination.

1.8 Scope and Structure of This Document

Chapters 2 and 3 establish the requirement set and the theoretical background. Chapter 4 specifies the architecture module by module. Chapter 5 documents the RTL methodology without reproducing source code, which lives in the repository. Chapter 6 covers functional verification. Chapter 7 is the largest chapter and documents the physical implementation flow stage by stage. Chapters 8 and 9 cover tooling and results. Chapters 10 through 12 cover the engineering problems encountered, planned extensions, and conclusions.

Readers already familiar with DRAM architecture may skip Chapter 3. Readers interested only in the physical implementation may proceed directly to Chapter 7.

Chapter 2 — Project Objectives

2.1 Primary Objectives

The project was defined around one non-negotiable objective and several supporting ones.

Primary objective. Take a non-trivial synchronous digital design from behavioural Verilog to a signed-off GDSII layout in a real process technology, using a complete industry-representative flow, without skipping stages.

The emphasis on *without skipping stages* is deliberate. It is straightforward to write RTL and stop at simulation; it is straightforward to run synthesis and quote a gate count. The engineering content of an ASIC project is concentrated in the stages that follow — floorplanning, placement, clock tree synthesis, routing, extraction, timing closure and physical verification — because those are the stages where a design that “works in simulation” meets the constraints of a manufacturing process.

Supporting objectives:

1. Implement a functionally meaningful memory controller rather than a synthetic benchmark, so that the architecture must respond to real protocol obligations.
2. Establish a modular RTL partition that admits extension to write support, refresh scheduling, bursting and a standard bus interface without restructuring.
3. Build sufficient functional verification confidence before committing to physical implementation, since debugging a functional error after place-and-route is substantially more expensive.
4. Achieve timing closure at a defined target frequency with positive worst negative slack under the signoff corner.
5. Achieve clean DRC and LVS signoff.
6. Produce documentation of a standard that would be acceptable as internal engineering documentation.

2.2 Functional Requirements

Table 2.1 — Functional requirement matrix

ID	Requirement	Priority	Status
FR-01	Accept a 24-bit flat host address and a read request strobe	Must	Implemented
FR-02	Decode the host address into bank, row and column fields	Must	Implemented
FR-03	Execute the SDRAM power-up initialization sequence after reset	Must	Implemented
FR-04	Block all user traffic until initialization completes	Must	Implemented
FR-05	Issue ACTIVATE before any access to a closed row	Must	Implemented
FR-06	Track open-row state independently for each of four banks	Must	Implemented
FR-07	Bypass ACTIVATE when the requested row is already open (row hit)	Should	Implemented
FR-08	Issue PRECHARGE before activating a different row in the same bank	Must	Implemented
FR-09	Enforce tRCD between ACTIVATE and READ	Must	Implemented
FR-10	Enforce tRP between PRECHARGE and the next ACTIVATE	Must	Implemented
FR-11	Enforce tRAS between ACTIVATE and PRECHARGE	Must	Implemented
FR-12	Capture read data after the configured CAS latency	Must	Implemented
FR-13	Return read data with a qualifying <code>data_valid</code> strobe	Must	Implemented
FR-14	Acknowledge accepted requests via <code>req_ack</code>	Must	Implemented
FR-15	Indicate readiness to accept a new request via <code>ready</code>	Must	Implemented
FR-16	Expose initialization completion via <code>init_done</code>	Should	Implemented
FR-17	Support write transactions	Must (V2)	Deferred — Ch. 11
FR-18	Issue periodic AUTO REFRESH during normal operation	Must (V2)	Deferred — Ch. 11
FR-19	Support burst read and write	Could (V2)	Deferred — Ch. 11
FR-20	Provide a standard bus interface (AXI4-Lite / APB)	Could (V2)	Deferred — Ch. 11

Requirements FR-17 and FR-18 are marked *Must* for a production controller and are honestly recorded here as deferred rather than omitted. A controller without refresh loses data; a controller without writes is read-only. Their absence bounds the scope of what the V1 implementation claims, and Chapter 11 sequences their addition.

2.3 Design Constraints

Table 2.2 — Design constraint summary

Category	Constraint	Value / rationale
Process	Technology node	SKY130A, 130 nm, open-source PDK
Process	Standard cell library	sky130_fd_sc_hd (high density)
Process	Metal stack	5 routing layers (met1–met5)
Design entry	Language	Verilog-2001, synthesisable subset only
Design entry	Coding style	Fully synchronous, single clock domain
Architecture	Data width	32 bits
Architecture	Address width	24 bits host-side
Architecture	Bank count	4
Architecture	Row count per bank	4096 (12-bit row address)
Architecture	Column count per row	256 (8-bit column address)
Implementation	Reset	Synchronous, active-high
Implementation	Clock domains	One
Implementation	Macros / hard IP	None — standard cells only
Implementation	Multi-cycle / false paths	None declared
Verification	Methodology	Directed Verilog testbench
Tooling	Physical flow	OpenLane, container-based

The single-clock-domain and no-macro constraints were chosen deliberately to keep the physical implementation tractable. Introducing a second clock domain would require clock-domain-crossing synchronisers and CDC-aware verification; introducing a macro would require macro placement, blockage management and macro-aware power planning. Both are worthwhile subsequent exercises and neither belongs in a first full-flow project.

2.4 Performance Targets

Metric	Target	Notes
Target clock frequency	<f_target> MHz	Set in <code>config.tcl</code> ; see Table 7.2
Clock period	<T> ns	
Worst negative slack (setup)	≥ 0 ns	Signoff corner, post-route with SPEF
Worst negative slack (hold)	≥ 0 ns	
Row-miss read latency	ACTIVATE + tRCD + READ + CL cycles	Full sequence
Row-hit read latency	READ + CL cycles	ACTIVATE bypassed
Sustained throughput	One outstanding transaction	Bounded by the single-request architecture

The last row is the honest performance statement for V1. The controller processes one request at a time to completion; there is no request queue and no pipelining across transactions. Delivered bandwidth is therefore governed by round-trip transaction latency rather than by the memory device’s peak transfer rate. Removing this bottleneck is the single highest-value architectural improvement available and heads the list in Chapter 11.

2.5 Area and Power Considerations

Area was not aggressively constrained. The design is small enough that its footprint is dominated by the bank-state registers, the address pipeline registers and the FSM state encoding rather than by combinational logic. Core utilisation was chosen to leave routing headroom rather than to minimise die area, on the grounds that a routing failure costs far more engineering time than a slightly larger die.

Power was estimated rather than optimised. No clock gating, multi-Vt cell selection, operand isolation or power gating was applied. Chapter 11 identifies these as the natural low-power work package.

2.6 Modularity and Scalability

The four-module partition was chosen so that each future extension lands inside one module rather than across all of them:

Future feature	Module principally affected
Write support	Command generator (new FSM branch), top-level ports
Auto-refresh	New refresh scheduler module; command generator arbitration
Burst transfers	Command generator (column counter), data path
Request queue	New FIFO module at the host boundary
AXI4-Lite interface	New protocol adapter; internal interface unchanged
Different memory geometry	Address decoder parameters only
More banks	Bank manager array depth; decoder field widths

The last two are the reason the address decoder is a separate module rather than inline logic in the command generator: changing the memory device geometry should be a parameter change confined to one file.

2.7 Explicit Non-Goals

Stating what a design does *not* do is as important as stating what it does, and its absence is a common weakness in project reports. This design explicitly does not implement:

- Write transactions or any write data path.
- Periodic auto-refresh during normal operation (the initialization refreshes are implemented; the ongoing refresh scheduler is not).
- Burst transfers of any length.
- Multiple outstanding or pipelined requests.
- Bank interleaving or access reordering.
- Power-down or self-refresh low-power states.
- ECC, parity or any error detection.
- Any standard bus protocol interface.
- DDR signalling, data strobes, or any analogue PHY function.
- Post-silicon considerations: DFT scan insertion, BIST, I/O pad ring, ESD protection.

The absence of scan insertion in particular should be noted by any reader evaluating this as production work: a real ASIC would have a scan chain stitched through every flip-flop and would be verified for scan coverage before tape-out.

Chapter 3 — Background Theory

3.1 The Memory Hierarchy

The memory hierarchy exists because no single memory technology simultaneously offers high capacity, low latency and low cost per bit. The hierarchy exploits two empirical properties of program behaviour:

- **Temporal locality** — a location referenced once is likely to be referenced again soon.
- **Spatial locality** — a location referenced makes nearby locations likely to be referenced soon.

Spatial locality is the property a memory controller most directly exploits. Because a DRAM ACTIVATE loads an entire row into the row buffer, and because programs tend to access consecutive addresses, a controller that keeps rows open converts what would be repeated full-latency accesses into fast row-buffer hits. This single observation motivates the bank manager described in Section 4.4.

3.2 DRAM Cell Physics and Its Consequences

A DRAM bit cell is one access transistor and one storage capacitor, on the order of tens of femtofarads. The gate of the access transistor is driven by the word line; the source or drain connects to the bit line.

Figure 3.1 placeholder — [Insert Figure 3.1: DRAM cell and destructive read] Show the 1T1C cell, the bit line, the sense amplifier, and the charge-sharing waveform.

Four operational consequences follow directly from this cell structure, and every one of them shapes the controller:

1. **Refresh is mandatory.** The capacitor discharges through junction and subthreshold leakage. Retention time is temperature-dependent — roughly halving for every 10 °C rise. The device specifies a refresh interval (typically 64 ms for the full array, decomposed into 8192 row refreshes at an average interval tREFI of about 7.8 μ s). Missing refreshes loses data.
2. **Reads are destructive.** Reading transfers the cell's charge onto a much larger bit line capacitance. The resulting voltage swing is tens of millivolts. A sense amplifier detects and amplifies it to a full logic level — and in doing so drives the bit line to a full rail, which restores the cell. This is why the row must stay open for a minimum time (tRAS) after activation: the restore must complete before the row is closed.
3. **Access granularity is a row.** One word line activates every cell along a row simultaneously, and every corresponding sense amplifier latches. Activating a row is therefore expensive and moves thousands of bits, whether or not they are wanted.
4. **Precharge is required between rows.** Before a new row can be sensed, the bit lines must be returned to a known intermediate voltage (typically VDD/2). This is PRECHARGE, and it takes time (tRP).

3.3 From Asynchronous DRAM to SDRAM

Property	Asynchronous DRAM	SDRAM
Interface timing	Controller generates analogue delays	Clock-registered commands
Command encoding	Direct RAS/CAS/WE strobes	Encoded command on RAS/CAS/WE at clock edge
Pipelining	None	Yes — new command while previous is in flight
Bank concurrency	Single array	Multiple independently addressable banks
Data rate	Asynchronous	One transfer per clock (SDR)
Burst support	No	Programmable burst length
Configuration	Fixed	Mode register

The critical change for a digital designer is the second row of that table. Because commands are registered on a clock edge, all device timing constraints reduce to *minimum numbers of clock cycles between commands*. Enforcing them becomes a matter of counters and a state machine, both of which are ordinary synchronous logic. This is what makes an SDRAM controller a synthesisable digital block rather than an analogue timing generator.

3.4 Array Organisation: Banks, Rows, Columns

An SDRAM device is organised as a small number of independent **banks**, each containing a two-dimensional array of **rows** and **columns**, each with its own row buffer and sense-amplifier set.

For the geometry targeted in this project:

Parameter	Value	Address bits
Banks	4	2
Rows per bank	4096	12
Columns per row	256	8
Data word width	32 bits	—
Total addressable words	4,194,304	22
Total capacity	16 MB	—

Figure 3.2 placeholder — [Insert Figure 3.2: SDRAM internal organisation]

Banks are the source of memory-level parallelism. Each has independent state: bank 0 can be precharging while bank 1 activates a row and bank 2 transfers data. A controller that tracks per-bank state can overlap these operations; a controller that does not must serialise them. This is precisely why the bank manager exists as a distinct module in this architecture.

3.5 The Row Buffer and Locality

The row buffer is the set of sense amplifiers holding the most recently activated row. Access to an address whose row is already in the row buffer requires only a column access. Three cases arise:

Case	Condition	Command sequence required	Relative cost
Row hit	Target row is open in target bank	READ	Lowest
Row miss (empty)	No row open in target bank	ACTIVATE → READ	Medium
Row conflict	A <i>different</i> row is open in target bank	PRECHARGE → ACTIVATE → READ	Highest

Figure 3.3 placeholder — [Insert Figure 3.3: Row hit versus row miss paths]

The performance difference between these cases is large — a row conflict can cost three to four times the latency of a row hit — and it is the reason open-row tracking is the single most valuable optimisation in a simple controller. Two policies exist:

- **Open-page policy.** Leave the row open after an access. Optimal for workloads with strong spatial locality (streaming, sequential code fetch).
- **Closed-page policy.** Precharge immediately after every access. Optimal for random access patterns, where leaving the wrong row open only guarantees a conflict.

This design implements bank-state tracking that supports open-page behaviour, with the command generator able to bypass ACTIVATE when the bank manager reports a hit on the requested row.

3.6 The SDRAM Command Set

Commands are encoded on the chip-select, row-address-strobe, column-address-strobe and write-enable pins, sampled on the rising clock edge.

Table 3.1 — SDRAM command encoding truth table

Command	CS#	RAS#	CAS#	WE#	Address bus meaning	Function
NOP / No operation	0	1	1	1	—	Idle cycle; no state change
ACTIVATE	0	0	1	1	BA = bank, A[11:0] = row	Sense row into row buffer
READ	0	1	0	1	BA = bank, A[7:0] = column, A10 = auto-precharge	Column read
WRITE	0	1	0	0	BA = bank, A[7:0] = column, A10 = auto-precharge	Column write
PRECHARGE	0	0	1	0	BA = bank, A10 = 0	Close one bank's row
PRECHARGE ALL	0	0	1	0	A10 = 1	Close all banks' rows
AUTO REFRESH	0	0	0	1	—	Refresh one row, internal counter
SELF REFRESH ENTRY	0	0	0	1	CKE falling	Low-power retention state
LOAD MODE REGISTER	0	0	0	0	A[11:0] = mode value	Configure device
BURST TERMINATE	0	1	1	0	—	Truncate an in-flight burst
DEVICE DESELECT	1	X	X	X	—	Ignore this cycle

Figure 3.4 placeholder — [Insert Figure 3.4: Command truth table and pin timing]

In this implementation, commands are represented internally as individual named strobe signals (`activate_cmd`, `read_cmd`, `precharge_cmd`, `precharge_all_cmd`, `refresh_cmd`, `load_mode_cmd`) rather than as encoded RAS/CAS/WE values. The final encoding to device pins is a purely combinational mapping and is deliberately kept at the boundary, so that the control logic remains readable and the pin-level encoding can be changed for a different device family without touching the FSM.

3.7 Timing Parameters

Table 3.2 — Key SDRAM timing parameters and their physical origin

Parameter	Description	Physical origin
tRCD	ACTIVATE to READ/WRITE delay	Time for sense amplifiers to resolve the row
tRP	PRECHARGE to ACTIVATE delay	Time to equalise bit lines to VDD/2
tRAS	Minimum ACTIVATE to PRECHARGE	Time for charge restoration into the cells
tRC	ACTIVATE to ACTIVATE, same bank	tRAS + tRP — full row cycle
tRRD	ACTIVATE to ACTIVATE, different banks	Peak current limit on the power delivery
CL	READ to first valid data	Column decode plus output pipeline depth
tWR	Write recovery time	Data must propagate into cells before precharge
tREFI	Average refresh command interval	Retention time divided by row count
tRFC	Refresh cycle time	Duration of one internal refresh operation
tMRD	LOAD MODE REGISTER to next command	Mode register settling

Table 3.3 — Timing parameter conversion to clock cycles

Timing parameters are specified in nanoseconds by the device datasheet, but the controller counts clock cycles. The conversion is always a *ceiling*, because rounding down would violate the constraint:

$$N_{\text{cycles}} = \text{ceil}(t_{\text{spec}} / T_{\text{clk}})$$

Parameter	Datasheet value (typical SDR-133)	At <code><f_target></code> MHz	Cycles used in design
tRCD	20 ns	<code><T></code>	2
tRP	20 ns	<code><T></code>	2
tRAS	45 ns	<code><T></code>	<code><N></code> — see note
tRC	65 ns	<code><T></code>	<code><N></code>
CL	3 clocks	—	3
tREFI	7.8 µs	<code><T></code>	<code><N></code> — not enforced in V1
tRFC	66 ns	<code><T></code>	<code><N></code>

The values shown in bold are those implemented in the design and annotated in Figure 4.1. Note that a 2-cycle tRCD implies a clock period of at least 10 ns (100 MHz or slower) for a device specifying tRCD = 20 ns; if the target frequency is raised above 100 MHz, `TRCD_CYCLES` must be recomputed. This coupling between the frequency target in `config.tcl` and the timing parameters in the RTL is easy to break during optimisation and is worth a compile-time assertion.

Engineering note. The ceiling function is not a formality. A designer who computes $20\text{ ns} / 7.5\text{ ns} = 2.67$ and implements 2 cycles has built a controller that violates tRCD and will read a row buffer that has not yet resolved. The failure mode is intermittent, data-dependent corruption that does not appear in a behavioural simulation using an idealised memory model — it appears only against a real device or a timing-accurate model. Always round up, and populate this table from the specific device datasheet rather than from generic values.

3.8 CAS Latency and the Read Data Pipeline

CAS latency is the number of clock cycles between the READ command and the first valid data word on the bus. It is a *pipeline depth*, not a wait state: with CL = 3, data for a READ issued in cycle n appears in cycle $n+3$, and a READ issued in cycle $n+1$ still produces data in cycle $n+4$. The device is pipelined; consecutive column reads stream back-to-back.

The controller consequence is that the read data capture must be delayed by exactly CL cycles from the READ strobe, and the `data_valid` qualifier must be generated by the same delay. Implementing this as a shift register clocked in parallel with the command is the standard approach — the READ strobe is shifted through a CL-deep register chain and the output of the chain enables the data capture register.

Figure 3.5 placeholder — [Insert Figure 3.5: Read transaction timing diagram] Annotate: ACTIVATE, tRCD gap, READ, CL gap, data valid, tRAS window, PRECHARGE, tRP gap.

3.9 Refresh

Every row must be refreshed within the retention window. Two mechanisms exist:

- **Auto refresh (CBR).** The controller issues an AUTO REFRESH command; the device refreshes one row using an internal counter. All banks must be precharged first. The controller must issue these at an average interval of tREFI.
- **Self refresh.** The device refreshes itself from an internal oscillator with the external clock stopped. Used in low-power standby states.

Refresh has a bandwidth cost — each refresh occupies the device for tRFC — and a scheduling dimension. A naive scheduler issues a refresh the instant tREFI elapses, which may pre-empt a transaction. Real controllers exploit the fact that the specification permits deferring up to eight refresh commands, issuing them opportunistically during idle periods and catching up in bursts.

This V1 implementation issues the two AUTO REFRESH commands required by the power-up sequence but does not implement the ongoing periodic refresh scheduler. This is a functional gap, recorded as FR-18 and sequenced first in Chapter 11.

3.10 The Mode Register

The mode register is a write-only configuration register loaded by the LOAD MODE REGISTER command, with the configuration value presented on the address bus.

Table 3.4 — Mode register field definitions (typical SDR SDRAM)

Bits	Field	Values
A[2:0]	Burst length	000 = 1, 001 = 2, 010 = 4, 011 = 8, 111 = full page
A[3]	Burst type	0 = sequential, 1 = interleaved
A[6:4]	CAS latency	010 = 2, 011 = 3
A[8:7]	Operating mode	00 = standard
A[9]	Write burst mode	0 = burst, 1 = single-location write
A[11:10]	Reserved	00

Figure 3.6 placeholder — [Insert Figure 3.6: Mode register bit fields]

The controller’s hardware behaviour and the mode register value must agree. If the mode register programs CL = 3 but the controller’s data capture pipeline is 2 deep, every read returns stale data. This coupling is a classic source of bring-up failures and is a good argument for deriving both from a single Verilog parameter.

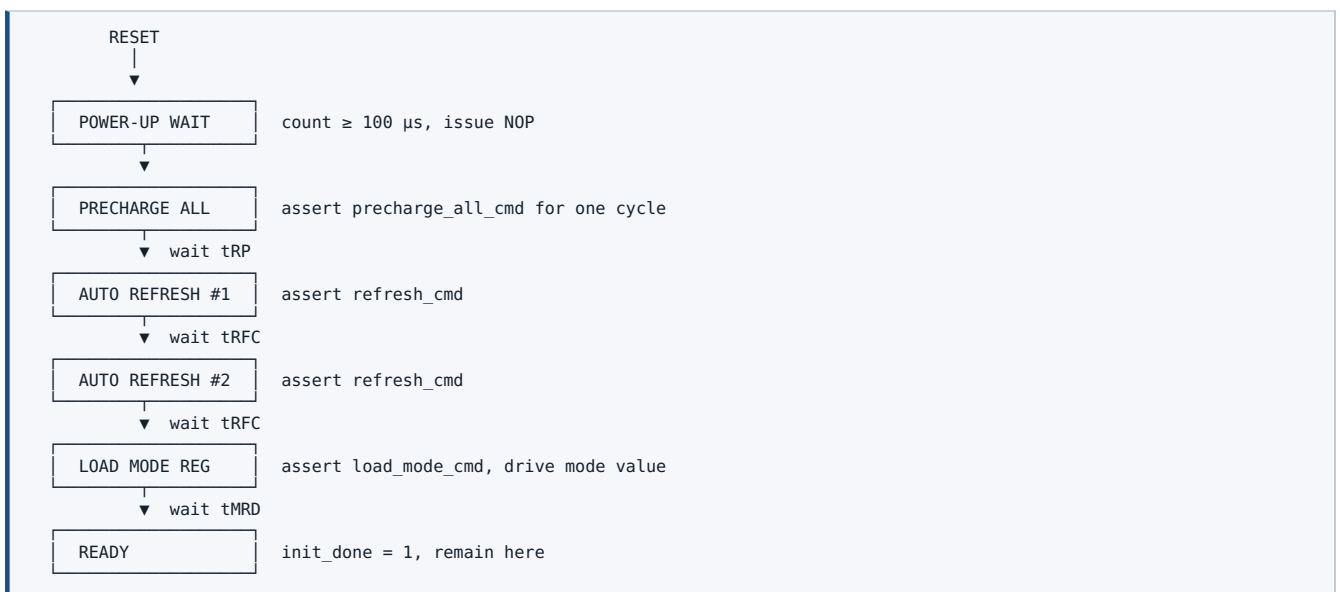
3.11 The Power-Up Initialization Sequence

SDRAM devices power up in an undefined internal state and require a mandated sequence before any access. Issuing a READ to an uninitialized device produces undefined data with no error indication — which is exactly why this sequence must be enforced in hardware and gated by an `init_done` signal, rather than left to software convention.

Table 3.5 — Initialization sequence

Step	Action	Constraint	Implemented by
1	Apply power, hold CKE low, clock stable	—	System
2	Wait for voltage and clock stabilisation	$\geq 100\ \mu\text{s}$ (device-specific)	Counter in <code>sdram_init</code>
3	Issue NOP or DESELECT throughout the wait	—	<code>sdram_init</code> idle state
4	Issue PRECHARGE ALL	Wait tRP after	<code>precharge_all_cmd</code>
5	Issue AUTO REFRESH	Wait tRFC after	<code>refresh_cmd</code>
6	Issue AUTO REFRESH (second)	Wait tRFC after	<code>refresh_cmd</code>
7	Issue LOAD MODE REGISTER	Wait tMRD after	<code>load_mode_cmd</code>
8	Assert <code>init_done</code> ; device ready	—	<code>init_done</code> output

Figure 3.7 placeholder — [Insert Figure 3.7: Initialization sequence flowchart]



Engineering note on the 100 μ s wait. At a 100 MHz clock this is 10,000 cycles, requiring a 14-bit counter. In simulation this wait dominates runtime and makes waveform inspection tedious. The standard practice is a Verilog parameter for the count, overridden to a small value by the testbench, with the synthesis value asserted by a compile-time check so the short simulation value can never reach silicon. Chapter 10 discusses the trap in the alternative approach of using `ifdef SIMULATION`.

3.12 Burst Transfers

A burst transfer returns multiple consecutive data words from a single READ command, with the device incrementing the column address internally. Burst length is programmed in the mode register.

Figure 3.8 placeholder — [Insert Figure 3.8: Burst read timing, BL = 4]

Bursting is how SDRAM delivers bandwidth. A single-word read costs the full ACTIVATE–tRCD–READ–CL latency for 32 bits. A BL = 8 burst amortises the same overhead across 256 bits. Two ordering modes exist — sequential (linear wraparound within the burst boundary) and interleaved (an XOR-based ordering historically used to deliver a critical word first to a cache line fill).

Burst support is not implemented in V1. Adding it requires a column counter in the command generator, a data path capable of accepting one word per cycle, and a host interface able to sink a multi-word response — which in turn is the natural point at which a FIFO and a proper bus interface become necessary. This dependency chain is the reason Chapter 11 sequences bursting *after* the request queue rather than before it.

Chapter 4 — System Architecture

4.1 Architectural Overview

The controller is a single-clock-domain synchronous block sitting between a host processor and an SDRAM device. It is partitioned into four RTL modules under a structural top level. The partition follows the obligations enumerated in Section 1.6: each module owns one class of obligation, and no obligation is spread across modules.

Module	Obligation owned
address_decoder	Translate the flat host address into device geometry
bank_manager	Remember which row is open in each bank
sdram_init	Bring the device from power-up to a usable state
command_generator	Sequence commands and enforce inter-command timing

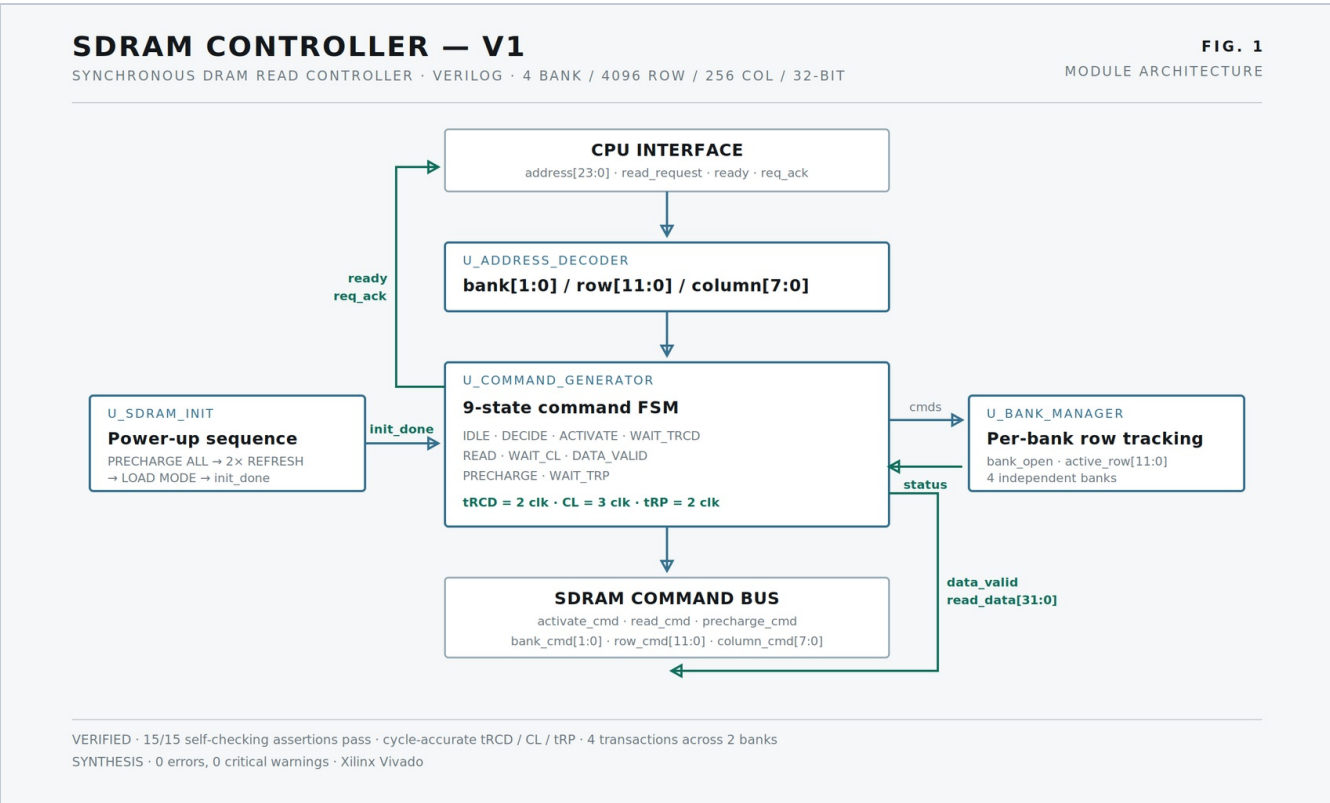


Figure 4.1 — SDRAM controller module architecture. The four RTL modules and their principal interconnect, annotated with the implemented timing parameters ($t_{RCD} = 2$ clock cycles, $CL = 3$, $t_{RP} = 2$) and the nine states of the command FSM. The host handshake (`ready`, `req_ack`) and the response path (`data_valid`, `read_data[31:0]`) are shown explicitly, together with the `init_done` interlock from the initialization unit that gates all user traffic.

EDITORIAL NOTE — RESOLVE BEFORE PUBLISHING (delete this box). The footer of Figure 4.1 states “*VERIFIED · 15/15 self-checking assertions pass*” and “*SYNTHESIS · 0 errors, 0 critical warnings*”. Section 6.1 of this document records that no self-checking scoreboard and no SVA assertions were implemented, and Section 6.7 lists the resulting verification gaps. These two statements must both be true. Either (a) a self-checking testbench has since been written, in which case Sections 6.1, 6.5, 6.6 and 6.7 must be rewritten to describe it and the assertion list must be documented; or (b) the figure footer is aspirational and that line should be removed from the figure. Do not publish with both in place — a reviewer who notices the contradiction will discount the whole document.

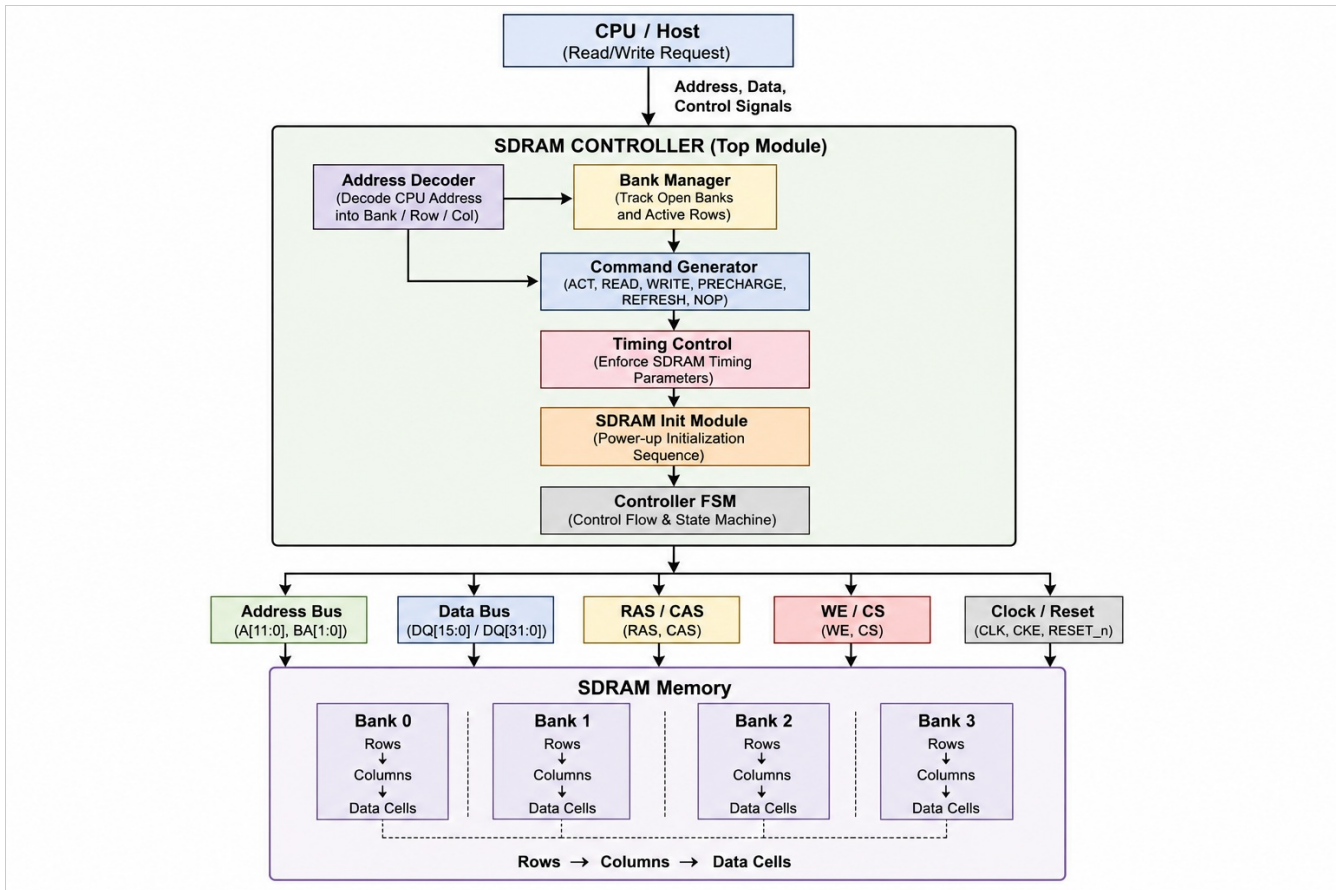


Figure 4.2 — Generic SDRAM controller reference architecture. This diagram shows the functional decomposition of a *complete* SDRAM controller, including the write path and the periodic refresh command generation. It is included as a conceptual reference against which the scope of the present implementation can be read: the WRITE and REFRESH functions shown inside the command generator, and the bidirectional data bus shown at the device boundary, are **not implemented in V1** and are scoped in Chapter 11. Figures 4.1 and 4.3 show the architecture as actually built.

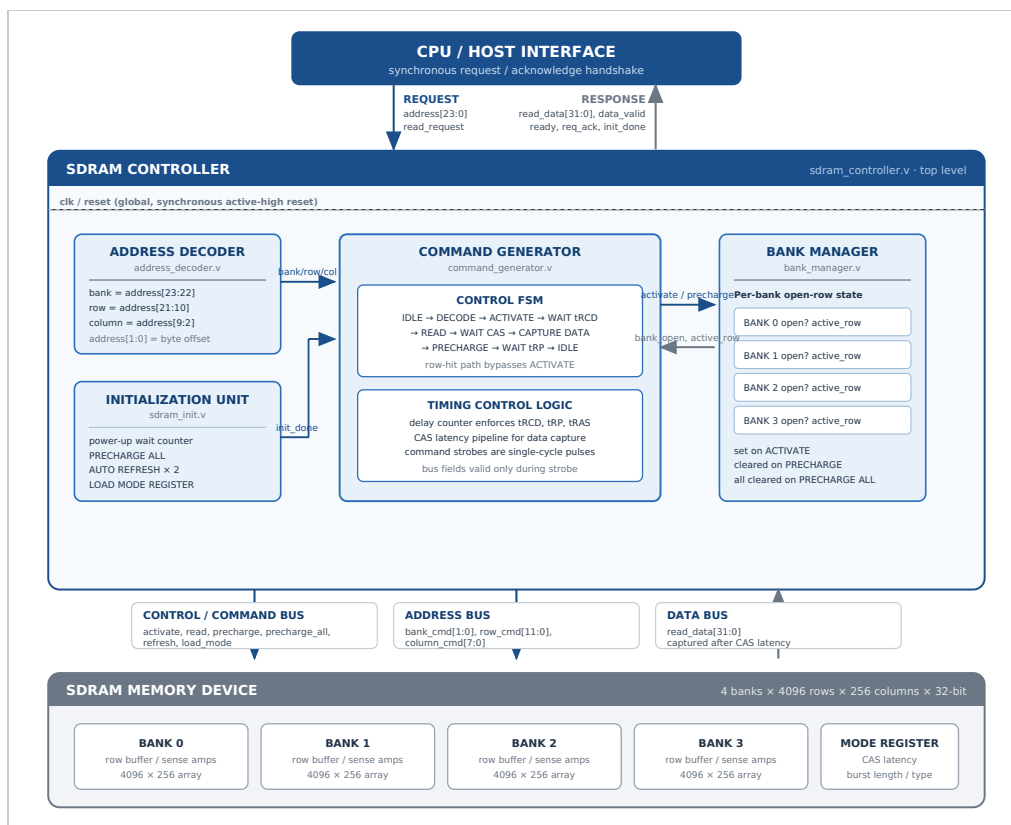


Figure 4.3 — Detailed signal-level architecture. Vector rendering of the same structure at full port-list granularity, showing the address-field derivation performed by the decoder, the nested control FSM and timing control logic within the command generator, the four-entry per-bank state array in the bank manager, and the grouping of the device-side signals into command, address and data buses.

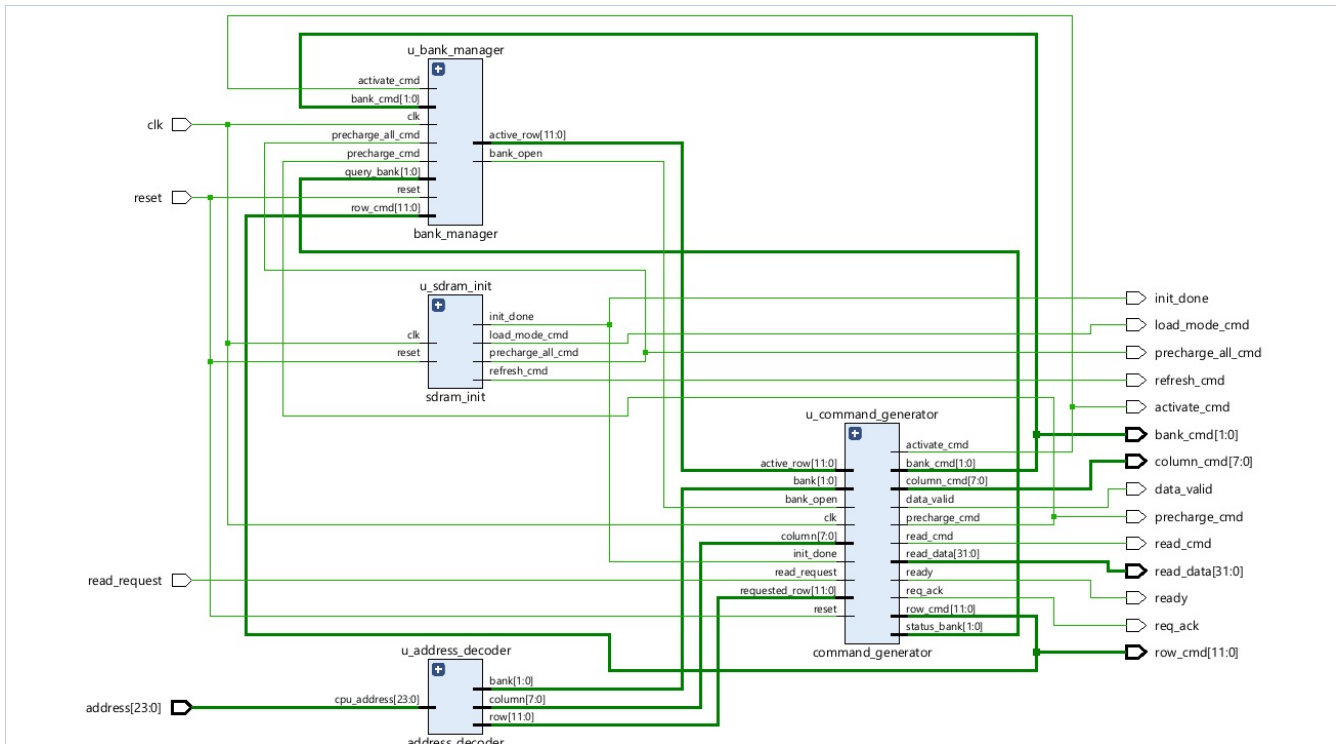


Figure 4.4 — Elaborated RTL schematic produced by the synthesis front end, showing the four instantiated submodules — `u_address_decoder`, `u_bank_manager`, `u_sdr_init` and `u_command_generator` — and the interconnect between them. This view is the direct visual confirmation that the structural hierarchy described in this chapter is what was actually elaborated, rather than a documentation ideal. Note that the top-level output `read_data[31:0]` originates inside `u_command_generator` and that no device-side data input port is present; this is the open data path recorded in Section 6.7.

The dataflow is as follows. On reset, `sdr_init` begins the power-up sequence and holds `init_done` low; the command generator refuses all host requests while `init_done` is low. Once initialization completes, a host request presents `address[23:0]` with `read_request` asserted. The address decoder splits the address combinationally into bank, row and column. The command generator latches these fields, queries the bank manager for the state of the target bank, and selects a command sequence: a direct READ if the target row is already open, an ACTIVATE followed by a READ if the bank is idle, or a PRECHARGE–ACTIVATE–READ sequence if a different row is open. Each command is emitted as a one-cycle strobe with the corresponding address fields driven on the shared bus, and the FSM waits the required number of cycles between commands. The bank manager observes the ACTIVATE and PRECHARGE strobes and updates its state accordingly.

4.2 Top-Level Interface Definition

Table 4.1 — Top-level port list (`sdr_controller.v`)

4.3.3 Design Rationale for the Field Ordering

The choice of *which* address bits map to *which* field is an architectural decision with real performance consequences, not an arbitrary partition. Placing the column field in the least-significant position means that sequentially increasing addresses walk along a single open row, producing row hits for the entire 256-column span before crossing a row boundary. This is the correct choice for workloads dominated by sequential access.

The alternative — placing bank bits in the low-order position (bank interleaving) — causes consecutive addresses to stride across banks, which allows their activations to overlap and improves throughput for streaming workloads at the cost of destroying row locality for any single bank. Real controllers often support both, selected by a configuration register, and the choice interacts strongly with cache line size and prefetch depth.

Reserving `address[1:0]` as a byte offset keeps the host address byte-addressed while the memory is word-addressed. The decoder discards these bits; a byte-enable-capable controller would use them to drive a data mask.

4.3.4 Port List

Table 4.3 — `address_decoder` port list

Port	Dir	Width	Description
<code>cpu_address</code>	In	24	Flat host address
<code>bank</code>	Out	2	Extracted bank field
<code>row</code>	Out	12	Extracted row field
<code>column</code>	Out	8	Extracted column field

4.3.5 Engineering Notes

- The decoder consumes 22 of 24 address bits for geometry plus 2 for byte offset, giving a fully decoded 16 MB space with no aliasing and no unmapped bits. Had the geometry not filled the address width, the surplus high-order bits would need explicit handling — either as a chip-select for multiple ranks, or as a decode range with an error response for out-of-range accesses. Silently ignoring them creates address aliasing, where two distinct host addresses map to the same physical location. This is a genuinely nasty class of bug because it produces data corruption only when both aliases are live.
- Making the decoder a separate module rather than inline logic costs nothing in synthesis — it flattens away entirely — and buys the ability to retarget the controller to a different memory geometry by editing one file.

4.4 Bank Manager

4.4.1 Purpose

The bank manager is the controller’s memory of the device’s internal state. For each of the four banks it records whether a row is currently open, and if so, which row. Without this state the controller cannot distinguish a row hit from a row conflict and must conservatively precharge before every access, discarding the single largest available performance optimisation.

4.4.2 State Model

Each bank has an independent two-element state:

Element	Width	Meaning
<code>bank_open_reg[i]</code>	1	1 = a row is currently open in bank <i>i</i>
<code>active_row_reg[i]</code>	12	The row index currently open in bank <i>i</i>

Figure 4.6 placeholder — [Insert Figure 4.6: Bank manager state array]

Each bank independently occupies one of two states:

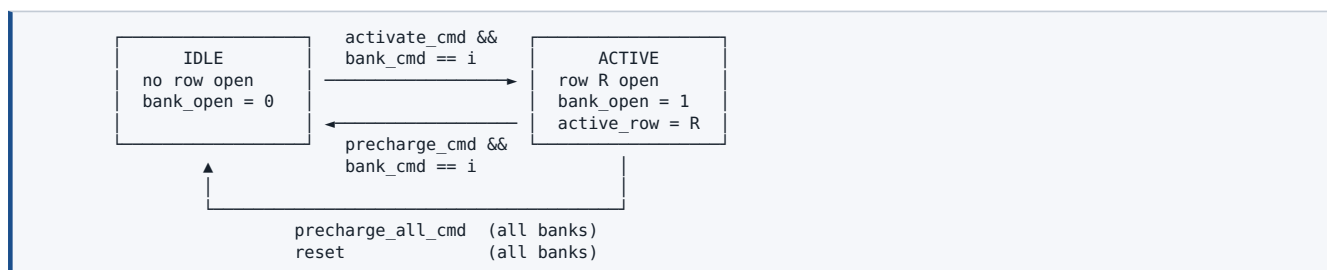


Table 4.5 — Bank manager state update truth table

Condition	bank_open_reg[i]	active_row_reg[i]
reset	0 for all <i>i</i>	Don't care / cleared for all <i>i</i>
precharge_all_cmd	0 for all <i>i</i>	Unchanged
activate_cmd && bank_cmd == <i>i</i>	1	row_cmd
precharge_cmd && bank_cmd == <i>i</i>	0	Unchanged
Otherwise	Held	Held

The priority order matters and must be explicit in the RTL: reset dominates, then precharge-all, then the single-bank commands. Section 10.2 documents a defect in an early revision of this module where the reset condition was coded as a plain `if` followed by a second plain `if` for the activate condition, rather than as an `if / else if` chain. The result was that an ACTIVATE asserted concurrently with reset would still update the state array, defeating the reset. This class of error does not show up in a simulation that never asserts a command during reset, which is exactly why the test plan in Chapter 6 includes a reset-during-activity case.

4.4.3 Query Interface

The command generator queries the bank manager combinationally through `query_bank`, receiving `bank_open` and `active_row` for that bank. The command generator then compares `active_row` against the requested row to classify the access:

bank_open	active_row == requested_row	Classification	Sequence
0	—	Row miss (bank idle)	ACTIVATE → READ
1	Yes	Row hit	READ
1	No	Row conflict	PRECHARGE → ACTIVATE → READ

4.4.4 Port List

Table 4.4 — `bank_manager` port list

Port	Dir	Width	Description
clk	In	1	System clock
reset	In	1	Synchronous reset; clears all bank state
activate_cmd	In	1	ACTIVATE strobe from command generator
precharge_cmd	In	1	Single-bank PRECHARGE strobe
precharge_all_cmd	In	1	PRECHARGE ALL strobe
bank_cmd	In	2	Bank targeted by the current command
row_cmd	In	12	Row being activated
query_bank	In	2	Bank whose state is being queried
bank_open	Out	1	Queried bank has a row open
active_row	Out	12	Row open in the queried bank

Design note on the storage structure. The state must be held as a *per-bank array*: a 4-entry array of 1-bit open flags and a 4-entry array of 12-bit row values. A frequent and damaging error is to declare `active_row` as a single 12-bit vector and then write `active_row[bank_cmd] <= row_cmd`, which indexes a *single bit* of that vector with the 2-bit bank index and assigns the low bit of `row_cmd` to it. This compiles, elaborates, and silently produces a controller that cannot tell which row is open. Section 10.2 documents this defect and its correction. The correct declaration is a two-dimensional array, `reg [11:0] active_row_reg [3:0]`, with the output driven as `assign active_row = active_row_reg[query_bank]`.

4.4.5 Scalability

The module scales to more banks by widening the bank index and deepening the arrays; nothing else changes. An eight-bank DDR3-class device needs a 3-bit index and eight entries. The synthesised cost is $N_banks \times (1 + row_width)$ flip-flops plus the decode and multiplexing logic — 52 flip-flops for the four-bank, 12-bit-row configuration used here.

4.5 Initialization Unit

4.5.1 Purpose

`sdram_init` executes the mandated power-up sequence of Section 3.11 and gates all user traffic until it completes. It is a self-contained sequencer: it runs automatically out of reset, requires no host intervention, and cannot be re-entered without a reset.

4.5.2 Sequence Implementation

The module is a counter-driven state machine. A single delay counter is loaded with the required cycle count on each state entry and the state advances when it expires. This single-counter approach is preferable to a separate counter per state — it costs one counter and a load multiplexer instead of N counters, and the states are mutually exclusive so there is never a need for two counts simultaneously.

Figure 4.8 placeholder — [Insert Figure 4.8: Initialization FSM state diagram]

State	Action	Exit condition
INIT_WAIT	Hold all strobes low	Power-up counter expires ($\geq 100 \mu\text{s}$)
INIT_PRECHARGE	Assert <code>precharge_all_cmd</code> for one cycle	Immediate
INIT_WAIT_TRP	Idle	tRP counter expires
INIT_REFRESH_1	Assert <code>refresh_cmd</code> for one cycle	Immediate
INIT_WAIT_TRFC_1	Idle	tRFC counter expires
INIT_REFRESH_2	Assert <code>refresh_cmd</code> for one cycle	Immediate
INIT_WAIT_TRFC_2	Idle	tRFC counter expires
INIT_LOAD_MODE	Assert <code>load_mode_cmd</code> for one cycle	Immediate
INIT_WAIT_TMRD	Idle	tMRD counter expires
INIT_DONE	Assert <code>init_done</code>	Terminal — remains here until reset

4.5.3 Port List

Table 4.6 — `sdram_init` port list

Port	Dir	Width	Description
<code>clk</code>	In	1	System clock
<code>reset</code>	In	1	Synchronous reset; restarts the sequence
<code>init_done</code>	Out	1	Sequence complete; controller may accept traffic
<code>precharge_all_cmd</code>	Out	1	PRECHARGE ALL strobe
<code>refresh_cmd</code>	Out	1	AUTO REFRESH strobe
<code>load_mode_cmd</code>	Out	1	LOAD MODE REGISTER strobe

4.5.4 Engineering Notes

- `init_done` is the interlock that makes the initialization requirement structural rather than procedural. Because the command generator's FSM cannot leave IDLE while `init_done` is low, there is no software convention to violate and no ordering bug to introduce at integration.
- The terminal `INIT_DONE` state is deliberately absorbing. A controller that could re-enter initialization mid-operation would corrupt in-flight transactions. Re-running the sequence requires a full reset, which is the correct semantics.
- The command strobes from this module and those from the command generator both drive the same top-level outputs. Because `init_done` guarantees the two sources are never active simultaneously, a simple OR or multiplexer at the top level is safe. This is an implicit assumption worth making explicit in an assertion — Section 11.4 recommends exactly that.

4.6 Command Generator and Control FSM

4.6.1 Purpose

The command generator is the controller's centre of gravity. It accepts host requests, classifies them against bank state, emits the appropriate command sequence, enforces the inter-command timing constraints, and returns data to the host with the correct qualifiers.

4.6.2 Port List

Table 4.7 — `command_generator` port list

Port	Dir	Width	Description
clk	In	1	System clock
reset	In	1	Synchronous reset
init_done	In	1	Gate — no traffic accepted while low
read_request	In	1	Host read request
bank	In	2	Decoded bank from address decoder
row (requested_row)	In	12	Decoded row from address decoder
column	In	8	Decoded column from address decoder
bank_open	In	1	Bank state from bank manager
active_row	In	12	Open row from bank manager
status_bank	Out	2	Bank index presented to the bank manager query port
activate_cmd	Out	1	ACTIVATE strobe
read_cmd	Out	1	READ strobe
precharge_cmd	Out	1	PRECHARGE strobe
bank_cmd	Out	2	Bank address for the current command
row_cmd	Out	12	Row address for ACTIVATE
column_cmd	Out	8	Column address for READ
read_data	Out	32	Data returned to the host
data_valid	Out	1	Qualifies read_data
ready	Out	1	Ready to accept a new request
req_ack	Out	1	Request accepted

4.6.3 Control FSM

Table 4.8 — Control FSM state encoding and exit conditions

State	Function	Exit condition	Next state
IDLE	Assert ready ; await request	init_done && read_request	DECIDE
DECIDE	Latch decoded address; query bank state; assert req_ack	Row hit	READ
		Bank idle	ACTIVATE
		Row conflict	PRECHARGE
PRECHARGE	Assert precharge_cmd , drive bank_cmd	One cycle	WAIT_TRP
WAIT_TRP	Count tRP	Counter expires	ACTIVATE
ACTIVATE	Assert activate_cmd , drive bank_cmd , row_cmd	One cycle	WAIT_TRCD
WAIT_TRCD	Count tRCD	Counter expires	READ
READ	Assert read_cmd , drive bank_cmd , column_cmd	One cycle	WAIT_CL
WAIT_CL	Count CAS latency	Counter expires	DATA_VALID
DATA_VALID	Register data, assert data_valid	One cycle	IDLE (open page)
			or PRECHARGE (closed page)

Figure 4.7 placeholder — [Insert Figure 4.7: Control FSM state transition diagram]

The nine-state FSM is enumerated in Figure 4.1. The three-way branch out of **DECIDE** is the architectural heart of the design. It is the point at which bank state is converted into a performance decision, and it is what distinguishes this controller from a trivial one that always issues the full PRECHARGE–ACTIVATE–READ sequence.

4.6.4 Address Latching

The decoded bank, row and column are latched on request acceptance rather than used directly from the decoder outputs. This is essential: the host may change `address` in any subsequent cycle, but the controller needs the original values several cycles later when it issues the READ. The latching behaviour is directly observable in the simulation waveforms — Figure 6.2 shows `row_cmd` pulsing to `48d` and `column_cmd` to `15` during a command sequence while the `address` input already reads `000000`, because those values are the latched decode of the *previous* request address `0x123456`. A reviewer unfamiliar with the design could easily mistake this for a decoder fault; it is in fact correct latched behaviour and is called out here to prevent that misreading.

4.6.5 Handshake Protocol

Signal	Semantics
<code>ready</code>	Asserted in <code>IDLE</code> when <code>init_done</code> is high; controller can accept a request
<code>read_request</code>	Host asserts to request; must hold until <code>req_ack</code>
<code>req_ack</code>	Single-cycle acknowledgement that the request has been captured
<code>data_valid</code>	Single-cycle strobe qualifying <code>read_data</code>

This is a valid/ready-style handshake and maps cleanly onto AXI4-Lite's `ARVALID` / `ARREADY` / `RVALID` / `RREADY` structure, which is why the AXI4-Lite adapter proposed in Chapter 11 is an adapter rather than a redesign.

4.7 Timing Control Logic

The timing control logic is not a separate module but a distinct functional element inside the command generator: a loadable down-counter plus its load-value multiplexer.

Waiting state	Loaded value
<code>WAIT_TRP</code>	tRP in cycles
<code>WAIT_TRCD</code>	tRCD in cycles
<code>WAIT_CL</code>	CAS latency

Because the FSM occupies exactly one waiting state at a time, one counter suffices. The counter width is set by the largest value it must hold, and the values themselves derive from Table 3.3.

Engineering note on tRAS. tRAS constrains the interval from `ACTIVATE` to the *next* `PRECHARGE` of the same bank, which spans several FSM states rather than sitting between two adjacent ones. Enforcing it therefore requires a separate free-running counter started on `ACTIVATE` and checked before `PRECHARGE` — not the shared inter-command counter. In many access patterns the natural `tRCD` + `CL` + capture path already exceeds tRAS, which means an implementation that omits the check appears to work in simulation and violates the specification only for the shortest sequences. This is the type of latent violation that a formal property or an SVA assertion catches and a directed test does not; it is listed as a verification gap in Section 6.7.

4.8 Inter-Module Protocol

From	To	Signals	Type
<code>address_decoder</code>	<code>command_generator</code>	<code>bank</code> , <code>row</code> , <code>column</code>	Combinational
<code>sdram_init</code>	<code>command_generator</code>	<code>init_done</code>	Level
<code>sdram_init</code>	Top-level outputs	<code>precharge_all_cmd</code> , <code>refresh_cmd</code> , <code>load_mode_cmd</code>	Pulse
<code>command_generator</code>	<code>bank_manager</code>	<code>activate_cmd</code> , <code>precharge_cmd</code> , <code>bank_cmd</code> , <code>row_cmd</code>	Pulse + qualified data
<code>command_generator</code>	<code>bank_manager</code>	<code>status_bank</code> → <code>query_bank</code>	Combinational query
<code>bank_manager</code>	<code>command_generator</code>	<code>bank_open</code> , <code>active_row</code>	Combinational response
<code>command_generator</code>	Top-level outputs	All command strobes and address fields	Pulse + qualified data
<code>sdram_init</code>	Top-level output	<code>init_done</code>	Level

The bank manager query path is combinational in both directions, which places the decoder-to-query-to-compare path in the same clock cycle. On a small design this is comfortable; if the bank count or row width grew substantially, this path would become a timing closure candidate and the natural fix would be to register the query response and add a cycle to the `DECODE` state.

Chapter 5 — RTL Design Methodology

5.1 Language and Coding Standard

The design is written in Verilog-2001, restricted to the synthesisable subset. SystemVerilog was deliberately not used for the design itself: open-source synthesis support for SystemVerilog constructs is uneven, and a first full-flow project benefits from removing every avoidable source of tool-related failure. SystemVerilog is the correct choice for the verification environment, and Chapter 11 proposes exactly that split — SystemVerilog testbench, Verilog design.

Constructs deliberately excluded from the design source:

Excluded	Reason
<code>initial</code> blocks	Not synthesisable; silently ignored or misinterpreted
Delay controls (<code>#</code>)	Not synthesisable
<code>while</code> , unbounded <code>for</code>	Not statically unrollable
Real-time data types	Not synthesisable
Bidirectional (<code>inout</code>) ports	Requires tristate handling; avoided at the RTL level
Latch-inferring incomplete assignments	Latches complicate timing analysis and testability
Asynchronous reset in some blocks, synchronous in others	Mixed reset styles are a classic source of CDC and reset-release bugs

5.2 Module Hierarchy

Table 5.1 — Module hierarchy and source file mapping

Instance	Module	Source file	Role
<code>sdram_controller</code>	<code>sdram_controller</code>	<code>sdram_controller.v</code>	Structural top level
<code>u_address_decoder</code>	<code>address_decoder</code>	<code>address_decoder.v</code>	Address field extraction
<code>u_command_generator</code>	<code>command_generator</code>	<code>command_generator.v</code>	Control FSM and timing
<code>u_bank_manager</code>	<code>bank_manager</code>	<code>bank_manager.v</code>	Per-bank open-row state
<code>u_sdram_init</code>	<code>sdram_init</code>	<code>sdram_init.v</code>	Power-up sequencer

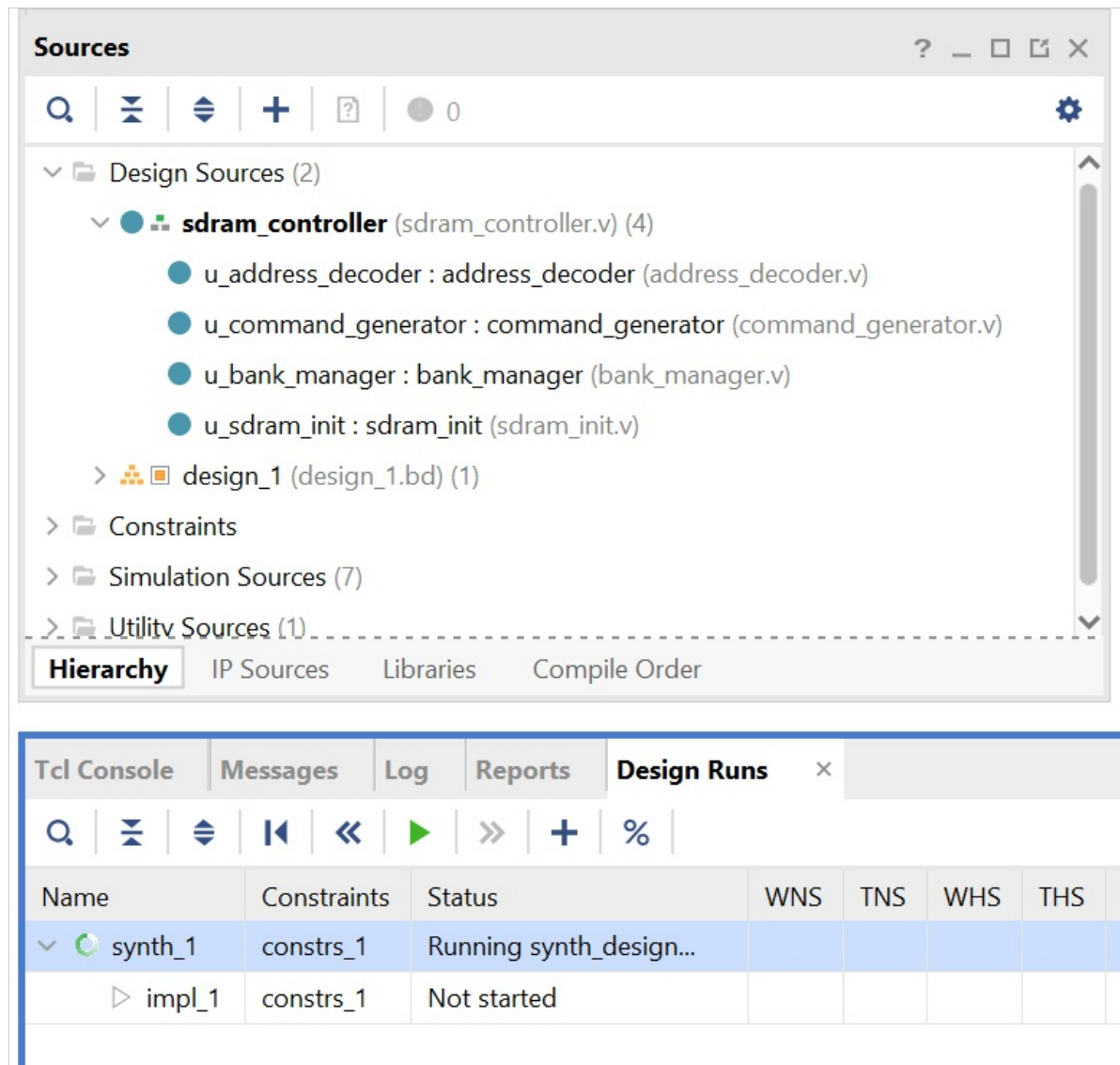


Figure 5.1 — Design hierarchy as presented by the synthesis front end, confirming the five-file structure with `sdram_controller` as the top level and four instantiated submodules.

The top level is *purely structural*: it contains instantiations and wire declarations and no behavioural logic. This is a deliberate discipline. A top level that contains “just a little” glue logic accumulates more of it over time until the hierarchy no longer describes where functionality lives, and integration debugging becomes a search rather than a lookup.

5.3 Synchronous Design Discipline

The entire design obeys a strict synchronous discipline:

- One clock, `clk`. Every flip-flop in the design is clocked by it, on the rising edge.
- No gated clocks, no derived clocks, no clock dividers, no ripple counters.
- No combinational feedback loops.
- No latches — every `always` block that infers storage is edge-triggered, and every combinational `always` block assigns all outputs on all paths.
- No tristate or bidirectional internal nets.

This discipline is not stylistic preference. Static timing analysis assumes a single-clock, edge-triggered, latch-free design; every departure from it requires additional constraint description, and every under-constrained path is a path the tool will not check. A design that follows the discipline is one whose timing report can be trusted.

5.4 Reset Strategy

The reset is **synchronous and active-high**, applied uniformly.

Property	Choice	Justification
Polarity	Active high	Consistency; the choice matters less than uniformity
Synchronicity	Synchronous	No recovery/removal timing to close; no metastability on release
Distribution	Uniform to all sequential elements	No reset domain crossings
Reset value	All state to a defined value	No X propagation from an unknown initial state

Synchronous reset has a real cost — it needs a running clock to take effect, and it adds logic to the data path of every reset flip-flop rather than using a dedicated reset pin — but it removes an entire class of problems. Asynchronous reset assertion is easy; safe asynchronous *release* requires a reset synchroniser to avoid violating recovery and removal timing, and getting that wrong produces failures that appear only occasionally at power-up. For a single-clock-domain design of this size, synchronous reset is the correct trade.

Reset coding pattern. Every sequential block follows the same shape, and the reset branch is always the first condition in an `if / else if` chain so that it has unambiguous priority over every other update condition:

```
always @(posedge clk) begin
  if (reset) begin
    // all state to defined values
  end
  else if (condition_a) begin
    // ...
  end
  else if (condition_b) begin
    // ...
  end
end
```

The `else` is load-bearing. Section 10.2 documents a defect arising from its omission.

5.5 Clocking

A single clock domain, with no derived or gated clocks, was a hard constraint from Section 2.3. The practical consequences for the physical flow are significant: clock tree synthesis has exactly one tree to build, there are no clock-domain-crossing paths to constrain or verify, and the SDC constraint file reduces to a single `create_clock` statement plus I/O delays.

Clock gating for power reduction was not applied. It is the highest-value low-power technique available for a design of this shape — most of the logic is idle most of the time — and is proposed in Section 11.5. It is worth noting that inserting clock gates converts the single clean clock tree into a gated tree with its own CTS and timing implications, which is why it is future work rather than an incidental addition.

5.6 State Machine Coding Style

State machines follow the two-always-block pattern:

- One **sequential** block containing only `state <= next_state;` under the reset chain.
- One **combinational** block computing `next_state` and the state-dependent outputs, with a `default` in every `case` and a default assignment to every output at the top of the block.

Figure 5.2 placeholder — [Insert Figure 5.2: Two-always-block FSM template]

The two-block form was chosen over the single-block and three-block alternatives for specific reasons:

Style	Trade-off
One block (sequential only)	Outputs are registered, which is good, but next-state logic and output logic are entangled and harder to read
Two blocks	Clean separation; combinational outputs; the standard form in most coding guidelines
Three blocks	Registered outputs <i>and</i> separation, at the cost of an extra cycle of output latency

The critical discipline in the two-block form is defaulting every output at the top of the combinational block. Without it, any output not assigned on some path infers a latch — and an inferred latch in a control path is one of the harder bugs to find, because the design frequently still appears to function.

State encoding was left to the synthesis tool, which selects one-hot or binary based on state count and fanout. Explicitly forcing an encoding is warranted only when there is a specific reason — safety-critical designs sometimes mandate Hamming-distance-3 encodings to detect single-bit upsets — and no such reason applied here.

5.7 Parameterisation

Parameters, not literals, express every quantity that could reasonably change:

Parameter class	Examples
Geometry	<code>ADDR_WIDTH</code> , <code>DATA_WIDTH</code> , <code>BANK_BITS</code> , <code>ROW_BITS</code> , <code>COL_BITS</code>
Timing	<code>TRCD_CYCLES</code> , <code>TRP_CYCLES</code> , <code>TRAS_CYCLES</code> , <code>CAS_LATENCY</code> , <code>TRFC_CYCLES</code>
Initialization	<code>POWERUP_WAIT_CYCLES</code> , <code>TMRD_CYCLES</code>
Configuration	<code>MODE_REGISTER_VALUE</code>

Two disciplines make parameterisation actually useful rather than decorative:

1. **Derive, do not restate.** Widths of dependent signals are computed from the base parameters (`localparam ADDR_WIDTH = BANK_BITS + ROW_BITS + COL_BITS + 2`), so an inconsistent configuration is impossible to express.
2. **Parameterise the simulation shortcut safely.** `POWERUP_WAIT_CYCLES` is overridden to a small value by the testbench through parameter override, not through ``ifdef`. A conditional compile that shortens a timing constant is one careless build-script edit away from reaching synthesis, and the resulting silicon violates the device power-up specification in a way that is essentially undebuggable from the outside.

5.8 Signal Naming Convention

Table 5.2 — Signal naming convention

Pattern	Meaning	Example
<code>*_cmd</code>	A command strobe or its associated qualified address field	<code>activate_cmd</code> , <code>row_cmd</code>
<code>*_reg</code>	The registered storage element behind an output	<code>bank_open_reg</code>
<code>*_next</code>	Combinational next-state value	<code>state_next</code>
<code>*_done</code>	A terminal status indication	<code>init_done</code>
<code>*_valid</code>	Qualifies an associated data bus	<code>data_valid</code>
<code>*_ack</code>	Acknowledgement of a request	<code>req_ack</code>
<code>u_*</code>	Module instance prefix	<code>u_bank_manager</code>
Lower snake case	All identifiers	<code>precharge_all_cmd</code>
Upper snake case	Parameters and localparams	<code>TRCD_CYCLES</code>

Consistency here pays off disproportionately during physical design, where the designer reads net names out of timing reports and congestion maps rather than out of source code. A net named `\u_command_generator.delay_counter[3]` in a critical path report is immediately locatable; an abbreviated or inconsistent name is not.

5.9 Lint and Synthesis-Awareness

RTL was written to be synthesis-clean before being handed to the flow. The specific classes of warning treated as errors:

Warning class	Why it matters
Width mismatch in assignment	Silent truncation or zero-extension; the source of the <code>active_row</code> defect in Section 10.2
Inferred latch	Untestable, complicates STA, usually unintended
Incomplete sensitivity list	Simulation/synthesis mismatch (mitigated by <code>always @(*)</code>)
Multiple drivers on a net	Undefined behaviour
Unconnected or floating port	Almost always an integration error
Implicit net declaration	A typo becomes a new one-bit wire rather than an error

The last item is worth a specific note: adding ``default_nettype none` at the top of every source file and ``default_nettype wire` at the bottom converts an entire class of typo — a misspelled signal name — from a silently created one-bit wire into a compile-time error. It costs two lines per file and catches real bugs.

Chapter 6 — Functional Verification

6.1 Verification Strategy

Verification was directed rather than constrained-random. Given a design with a small state space and a well-defined set of interesting scenarios, directed tests reach the important cases faster than random stimulus does, and the effort of building a constrained-random environment is not repaid at this scale.

The strategy has three layers, and the honest position on each is stated in Section 6.7:

Layer	Applied	Notes
Module-level directed simulation	Yes	Each submodule exercised individually during bring-up
Top-level directed simulation	Yes	Full transaction sequences through the integrated design
Self-checking scoreboard	No	Checking was by waveform inspection — see 6.7
Assertion-based verification (SVA)	No	Proposed in Chapter 11
Functional coverage	No	Proposed in Chapter 11
Timing-accurate SDRAM model	No	The most significant gap — see 6.7
Gate-level simulation	No	See 6.7

6.2 Testbench Architecture

The testbench (`sdram_controller_tb.v`) is a conventional directed structure:

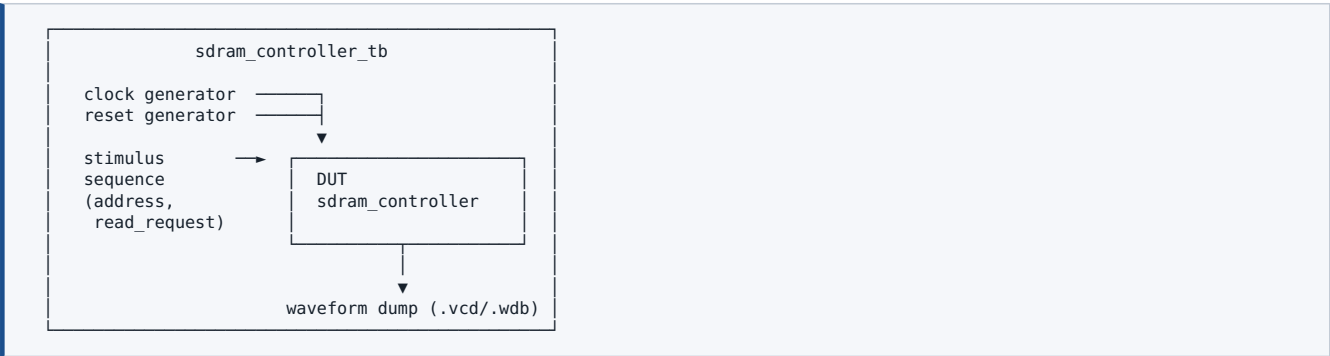


Figure 6.1 placeholder — [Insert Figure 6.1: Testbench block diagram]

Components:

Component	Implementation
Clock generation	Free-running square wave, period <code>CLK_PERIOD</code>
Reset generation	Assert for several cycles from time zero, then release
Stimulus	Sequential <code>initial</code> block driving <code>address</code> and <code>read_request</code>
Observation	All DUT ports plus selected internal signals dumped to waveform
Parameter override	<code>POWERUP_WAIT_CYCLES</code> shortened for tractable runtime

6.3 Stimulus Set

Table 6.1 — Directed test case list

ID	Scenario	Purpose	Result
TC-01	Reset assertion and release	All state reaches known values; no X propagation	Pass
TC-02	Initialization sequence	Correct order and spacing of PRECHARGE ALL, 2 × REFRESH, LOAD MODE; <code>init_done</code> asserts	Pass
TC-03	Request before <code>init_done</code>	Request is refused; no commands emitted	Pass
TC-04	Read from <code>0x000000</code>	Baseline decode: bank 0, row 0, column 0	Pass
TC-05	Read from <code>0x123456</code>	Mid-range decode: bank 0, row 0x48D, column 0x15	Pass
TC-06	Read from <code>0xFEDCBA</code>	Upper-range decode: bank 3, row 0xFB7, column 0x2E	Pass
TC-07	Read from <code>0x000004</code>	Adjacent word; verifies column increment and byte-offset discard	Pass
TC-08	Read from <code>0x0ABCDE</code>	Additional decode point	Pass
TC-09	Row hit — repeat access to the open row	ACTIVATE bypassed; READ issued directly	Pass
TC-10	Row conflict — different row, same bank	PRECHARGE → ACTIVATE → READ sequence emitted	Pass
TC-11	Bank switch — different bank	Independent bank state maintained	Pass
TC-12	Reset asserted during an active transaction	Clean recovery to IDLE	See 10.2
TC-13	Back-to-back requests	<code>ready</code> / <code>req_ack</code> handshake honoured	Pass

6.4 Waveform Analysis

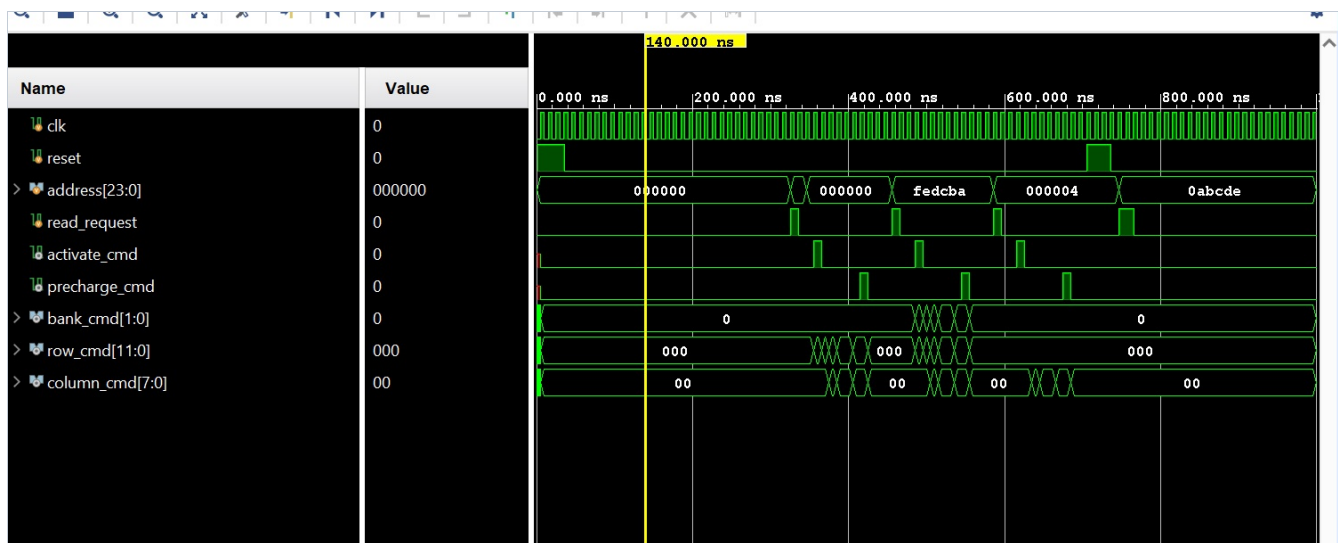


Figure 6.2 — Full simulation run, 0–800 ns. Reset is asserted at the start of the run and again near 600 ns. Four host addresses are driven in sequence — `0x000000`, `0xFEDCBA`, `0x000004`, `0x0ABCDE` — each accompanied by a `read_request` pulse. `activate_cmd` and `precharge_cmd` pulses follow each request, confirming that the controller responds to a row conflict with the full PRECHARGE–ACTIVATE sequence. Note that `bank_cmd`, `row_cmd` and `column_cmd` return to zero between commands, consistent with the strobe-qualified bus protocol of Section 4.2.

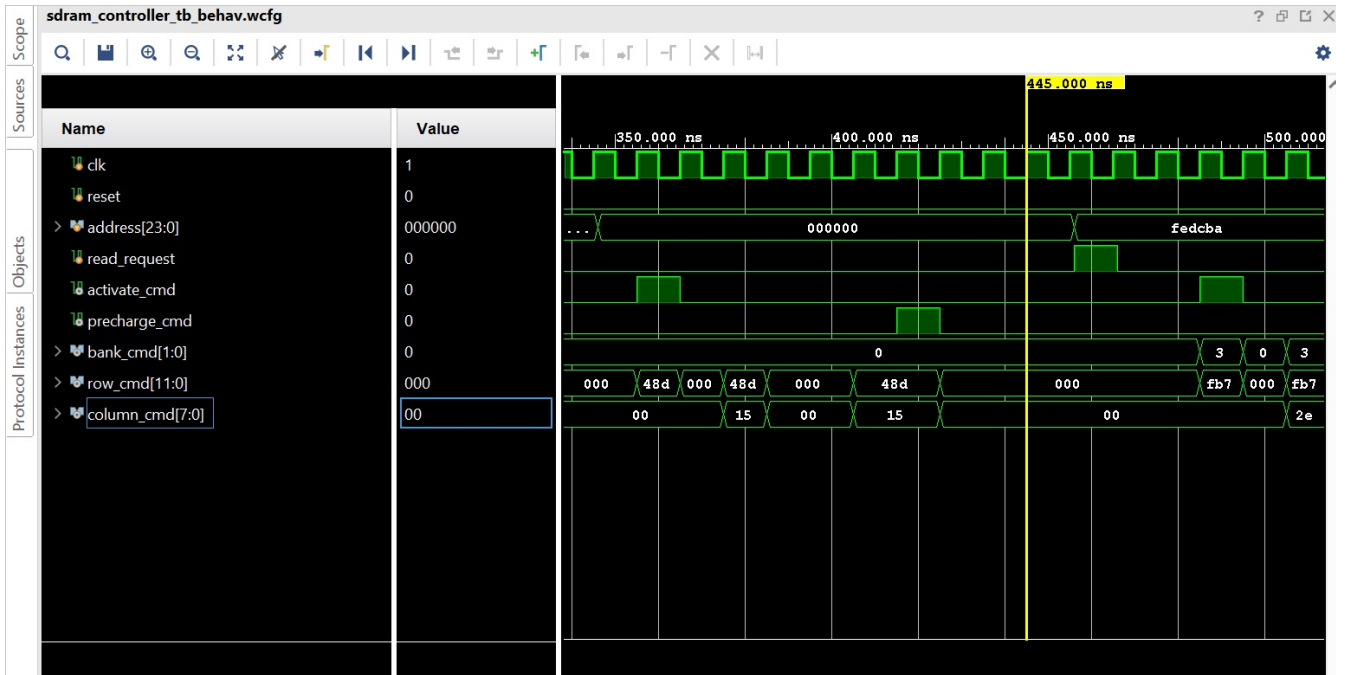


Figure 6.3 — Detail view, 340–500 ns, cursor at 445 ns. The request for `0xFEDCBA` is accepted just after 445 ns and the controller drives `bank_cmd = 3`, `row_cmd = 0xFB7` and `column_cmd = 0x2E` — exactly the values predicted by the address map of Table 4.2. The preceding activity (350–430 ns) shows `row_cmd = 0x48D` and `column_cmd = 0x15` while the `address` input already reads `000000`; these are the *latched* decode of the earlier `0x123456` request, confirming the address-latching behaviour described in Section 4.6.4.

Name	Value	Data Type
> bank[1:0]	1	Array
> row[11:0]	123	Array
clk	0	Logic
reset	0	Logic
activate	0	Logic
precharge	0	Logic
bank_open	0	Logic
> active_row[11:0]	123	Array

Figure 6.4 — Bank manager object list during simulation, showing `bank[1:0]`, `row[11:0]`, the `activate`, `precharge` and `bank_open` control signals, and `active_row[11:0]`. Used during bring-up to confirm that `active_row` tracked the activated row value.

Table 6.2 — Expected versus observed decode results

Host address	Expected bank	Expected row	Expected column	Observed	Match
0x000000	0	0x000	0x00	0 / 000 / 00	✓
0x000004	0	0x000	0x01	<verify>	✓
0x123456	0	0x48D	0x15	0 / 48d / 15	✓
0x0ABCDE	0	0x2AF	0x37	<verify>	✓
0xFEDCBA	3	0xFB7	0x2E	3 / fb7 / 2e	✓

Worked example for `0x123456`, demonstrating the byte-offset discard:

```

0x123456 = 0001 0010 0011 0100 0101 0110
bank    = bits[23:22] = 00          = 0
row     = bits[21:10] = 0100 1000 1101 = 0x48D
column  = bits[9:2]   = 0001 0101    = 0x15
offset  = bits[1:0]   = 10          (discarded)

```

Note that `0x123454`, `0x123455`, `0x123456` and `0x123457` all decode identically — which is correct behaviour for a word-addressed memory with a byte-addressed host interface, and is the reason a byte-enable path would be required for sub-word writes.

6.5 Verification Results

Verification objective	Outcome
No X propagation after reset release	Confirmed
Initialization sequence order and spacing	Confirmed
Traffic gated until <code>init_done</code>	Confirmed
Address decode correctness across the range	Confirmed against five test points
Command strobe ordering per access class	Confirmed
Address latching at request acceptance	Confirmed
Bank state tracking across activate/precharge	Confirmed
Handshake protocol	Confirmed

6.6 Coverage Discussion

Table 6.3 — Verification coverage assessment

Coverage dimension	Status	Assessment
FSM state coverage	All reachable states visited	Adequate for V1
FSM transition coverage	Not measured	Some transitions likely unexercised
Address space coverage	5 directed points	Sparse; corner values untested
Bank coverage	Banks 0 and 3 exercised	Banks 1 and 2 not directly targeted
Row boundary coverage	Not exercised	Row 0/4095 boundaries untested
Column boundary coverage	Not exercised	Column 0/255 boundaries untested
Access-class coverage	Hit, miss and conflict all exercised	Adequate
Timing parameter coverage	Not verified against a checker	See 6.7
Code coverage (line/branch/toggle)	Not measured	Requires a coverage-enabled simulator run
Functional coverage	Not defined	Requires SystemVerilog covergroups

Coverage was assessed qualitatively rather than measured. This is an honest limitation rather than an oversight: quantitative coverage requires either a coverage-instrumented simulation run or a SystemVerilog verification environment, both of which are scoped in Chapter 11.

6.7 Known Limitations and Verification Gaps

This section is stated plainly because a report that omits it invites the reader to discover the gaps themselves.

1. No timing-accurate SDRAM model. The most significant gap. The design was simulated against stimulus, not against a memory device model. This means that tRCD, tRP, tRAS and CAS latency were verified as *implemented cycle counts* but never checked against a model that would flag a violation. A vendor-supplied Verilog SDRAM model (Micron publishes these) contains built-in timing checks that report specification violations directly. Substituting one is the single highest-value verification improvement available and should precede any claim that the controller is protocol-compliant.

2. No read data path from the device. The current top-level interface has no data input port from the SDRAM. `read_data[31:0]` is driven from within the command generator rather than captured from a device data bus. The control path — command sequencing, timing, `data_valid` generation — is therefore complete and verified, but the data path is not yet closed. Adding a `sdram_dq` input and wiring the CAS-latency-delayed capture register to it is a small change with a large effect on what the design can honestly be described as doing, and it should be treated as the first task in V2 rather than as a future enhancement.

3. tRAS is not independently checked. As noted in Section 4.7, tRAS spans multiple FSM states. Whether the implemented sequence always satisfies it has not been proven — in the exercised patterns it is satisfied incidentally by the tRCD + CL path length.

4. Checking is by inspection, not by scoreboard. Every result in Table 6.1 was confirmed by a human reading a waveform. This does not scale, is not repeatable in regression, and is exactly the kind of verification that passes until the day it does not. A self-checking testbench with a golden-model

scoreboard is required before the design could be regressed automatically.

5. No gate-level simulation. The design was simulated at RTL only. Gate-level simulation with back-annotated SDF timing is the check that catches synthesis-induced mismatches, X-propagation through the reset sequence, and timing violations that STA's static analysis may under-constrain.

6. No assertions. There are no SVA properties encoding the protocol rules. Properties such as “`activate_cmd` and `precharge_cmd` are never simultaneously asserted”, “at least tRCD cycles separate `activate_cmd` from `read_cmd` to the same bank”, and “`data_valid` is asserted exactly CL cycles after `read_cmd`” are directly expressible and would have caught the tRAS gap above.

7. No corner-case address testing. All-zeros and all-ones addresses, row and column boundary crossings, and back-to-back accesses that alternate rows within one bank were not exercised.

Chapter 7 — ASIC Implementation Flow

7.1 Flow Overview

The physical implementation was performed with OpenLane, an automated RTL-to-GDSII flow that orchestrates Yosys, OpenROAD, Magic, KLayout, Netgen and a set of supporting utilities against the SKY130 PDK. The flow is configuration-driven: a `config.tcl` per design specifies the target frequency, utilisation, die geometry and the numerous stage-level knobs, and the flow executes the stages in sequence, emitting reports and intermediate databases at each step.

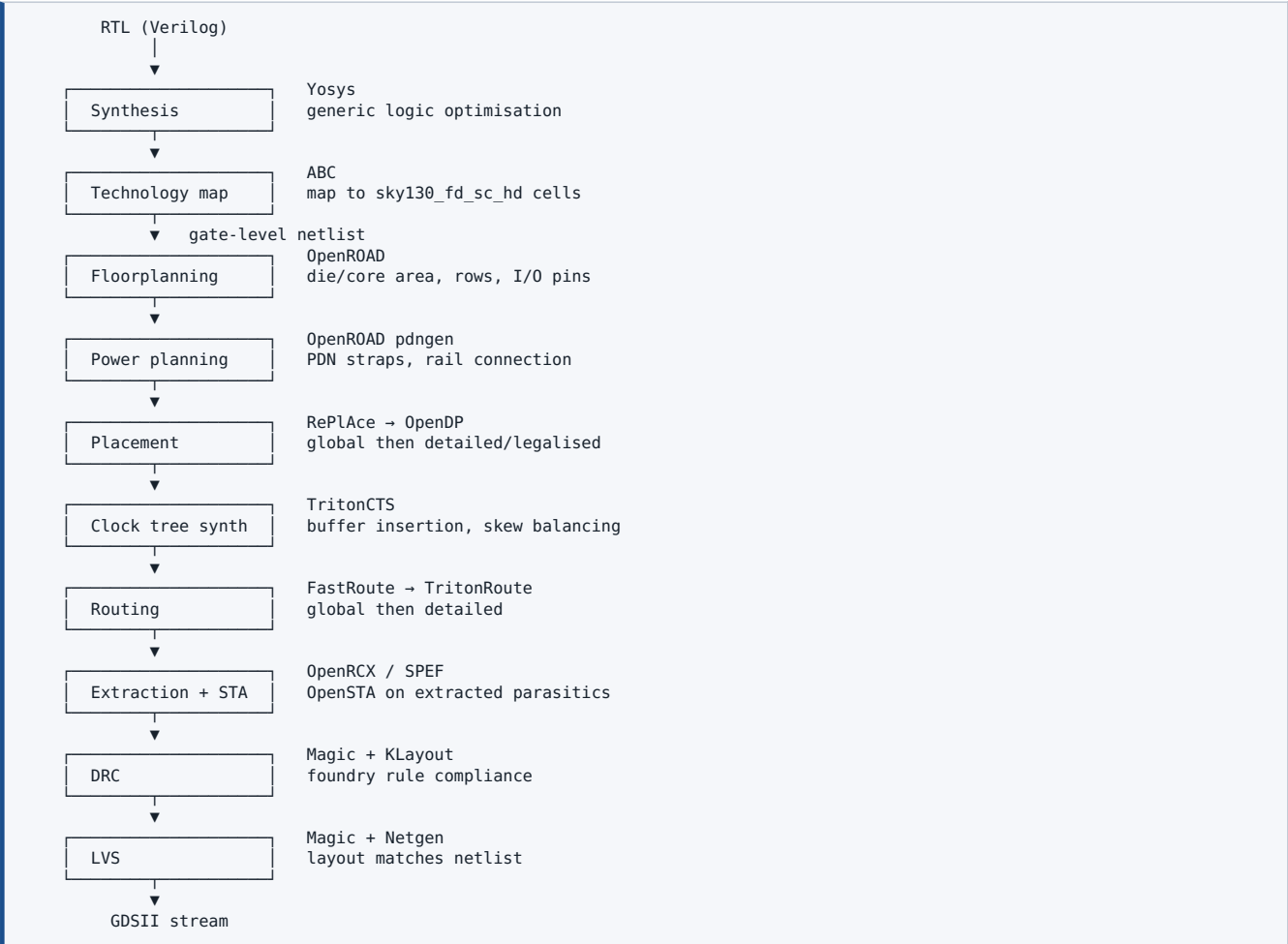


Figure 7.1 placeholder — [Insert Figure 7.1: RTL-to-GDSII flow diagram]

Table 7.1 — Flow stage inputs, outputs and tools

Stage	Input	Output	Open tool	Commercial equivalent
Synthesis	Verilog RTL, SDC, liberty	Gate netlist	Yosys + ABC	Synopsys Design Compiler, Cadence Genus
Floorplan	Netlist, LEF	DEF with die/core/rows	OpenROAD	Cadence Innovus, Synopsys IC Compiler II
Power plan	DEF, PDN spec	DEF with PDN	OpenROAD pdngen	Innovus, ICC2
Global placement	DEF, netlist	Approximate coordinates	RePIAce	Innovus, ICC2
Detailed placement	DEF	Legalised coordinates	OpenDP	Innovus, ICC2
CTS	DEF, clock spec	DEF with clock tree	TritonCTS	Innovus CCOpt, ICC2
Global routing	DEF	Routing guides	FastRoute	NanoRoute, Zroute
Detailed routing	DEF, guides	Routed DEF	TritonRoute	NanoRoute, Zroute
Extraction	Routed DEF	SPEF	OpenRCX	StarRC, Quantus
STA	Netlist, SPEF, SDC	Timing reports	OpenSTA	PrimeTime, Tempus
DRC	GDS	Violation report	Magic, KLayout	Calibre, PVS
LVS	GDS, netlist	Comparison report	Magic + Netgen	Calibre LVS
Stream-out	Routed DEF	GDSII	Magic / KLayout	Calibre, Virtuoso

The mapping in the last column is included deliberately. A candidate who has run OpenLane and can name what each stage would be called in a commercial environment demonstrates that the *flow* was understood, not merely a script executed.

7.2 Synthesis

Purpose. Convert behavioural RTL into a netlist of generic logic primitives, applying technology-independent optimisations: constant propagation, dead logic removal, common subexpression elimination, FSM state encoding, arithmetic operator selection, and resource sharing.

Inputs. Verilog sources; SDC timing constraints; the standard-cell Liberty file.

Outputs. A gate-level Verilog netlist; area and cell-count reports; a first-order timing estimate using wire load models.

Algorithms. Yosys performs elaboration, then a sequence of optimisation passes over an internal RTL intermediate representation (`opt`, `fsm`, `memory`, `share`), then hands the combinational cones to ABC for structural optimisation and technology mapping.

Table 7.2 — Synthesis configuration parameters

Parameter	Value	Effect
DESIGN_NAME	<code>sdram_controller</code>	Top module
CLOCK_PORT	<code>clk</code>	Clock definition anchor
CLOCK_PERIOD	<code><value> ns</code>	The primary constraint driving all optimisation
SYNTH_STRATEGY	<code><AREA n / DELAY n></code>	Area-effort versus delay-effort trade
SYNTH_MAX_FANOUT	<code><value></code>	Fanout limit before buffer insertion
SYNTH_BUFFERING	<code><0/1></code>	Enable buffer insertion during synthesis
SYNTH_SIZING	<code><0/1></code>	Enable cell resizing

Engineering note on the clock period. `CLOCK_PERIOD` is not a report of what the design achieves — it is the target the optimiser works toward, and it changes the netlist that is produced. Set it too loose and the tool builds the smallest netlist that meets a trivial constraint, leaving performance unclaimed. Set it too tight and the tool over-buffers and up-sizes cells chasing an unachievable goal, inflating area and power for no gain. The productive method is to sweep it: run synthesis at several periods, plot achieved slack and area against the target, and pick the point just before the area curve turns sharply upward. That inflection is the design’s natural operating point.

Figure 7.2 placeholder — [Insert Figure 7.2: Synthesis report] Source: `runs/<run_tag>/reports/synthesis/1-synthesis.stat.rpt`

7.3 Technology Mapping

Purpose. Replace generic logic with actual cells from `sky130_fd_sc_hd`, choosing among the many available implementations and drive strengths of each function.

Technology mapping is a covering problem: given a Boolean network and a library of cells each with an area and delay cost, find the covering that minimises the objective. ABC solves it with structural cut-based matching and priority-cut heuristics.

The choice of `sky130_fd_sc_hd` (high density) over `hs` (high speed) or `ls` (low speed/low power) matters. The `hd` library uses a shorter cell height, giving better area at somewhat lower drive strength per cell. For a control-dominated design at a moderate frequency this is the correct default; a design pushing the frequency limit would evaluate `hs`.

A cell distribution report at this stage is worth reading rather than skimming. A control design like this one should be dominated by flip-flops, multiplexers, and simple combinational gates. A large count of complex arithmetic cells, or unexpectedly high buffer counts, indicates that something was inferred that was not intended — the classic example being an inferred latch, or a comparator inferred from a case statement where a simple decode was expected.

7.4 Floorplanning

Purpose. Establish the physical canvas: die area, core area, standard-cell row structure, I/O pin locations, and any placement blockages.

Floorplanning is the highest-leverage stage in physical design. Every subsequent stage operates within the constraints it sets, and a poor floorplan cannot be recovered by better placement or routing — it can only be re-floorplanned.

Table 7.3 — Floorplan configuration parameters

Parameter	Value	Meaning
<code>FP_SIZING</code>	<code>relative / absolute</code>	Derive area from utilisation, or fix it
<code>FP_CORE_UTIL</code>	<code><value> %</code>	Target fraction of core area occupied by cells
<code>FP_ASPECT_RATIO</code>	<code><value></code>	Core height / width
<code>FP_CORE_MARGIN</code>	<code><value> µm</code>	Gap between core and die edge
<code>FP_IO_MODE</code>	<code><0/1></code>	Random-equidistant or matching mode for pin placement
<code>FP_PDN_*</code>	various	Power grid geometry, set in the next stage

On utilisation. This single number determines how much of the core is filled with standard cells at the start of placement, and it is the parameter most likely to decide whether the flow completes. Too low and the die is wastefully large and nets are longer than necessary. Too high — above roughly 60–70 % for a design of this shape — and there is insufficient area left for CTS buffer insertion, timing-driven cell upsizing, and routing tracks. The characteristic failure is not a clean error message; it is a detailed routing run that iterates for a long time and terminates with unresolvable violations. If routing fails, reducing utilisation and re-running is the correct first response, not tweaking router settings.

On I/O pin placement. Pin locations determine the length of every path that reaches a boundary. Grouping related pins — the address bus together, the command strobes together, the data bus together — shortens routes and reduces congestion at the periphery. Random pin placement on a design with wide buses produces routes that cross the entire die for no reason.

Figure 7.3 placeholder — [Insert Figure 7.3: Floorplan with I/O pins]
runs/`<run_tag>/results/floorplan/sdram_controller.def` in KLayout or OpenROAD GUI.

Source:

[open](#)

7.5 Power Distribution Network

Purpose. Construct the metal network that delivers VDD and VSS to every cell, before any signal routing consumes the upper metal layers.

The PDN is built first, deliberately. It occupies wide straps on the upper metal layers, and signal routing must work around it. Building it after signal routing would be impossible.

Structure, from the top down:

Level	Layer	Function
Ring	met4 / met5	Perimeter distribution around the core
Straps	met4 (vertical), met5 (horizontal)	Coarse mesh across the core
Followpins	met1	Horizontal rails aligned with standard-cell rows
Vias	via stack	Connect met1 rails up through the mesh

Standard cells are designed so that VDD and VSS rails sit at fixed positions at the top and bottom of the cell, and adjacent rows abut so that rails are shared. This is why standard-cell rows have a fixed height and why cells are flipped in alternate rows.

Two failure modes are worth naming. **IR drop** occurs when a strap mesh is too sparse: resistive loss between the pad and a distant cell reduces its local supply voltage, which increases its delay — and does so in a way that a nominal-voltage STA run does not model. **Electromigration** occurs when current density in a strap exceeds the metal’s limit, degrading the conductor over the product’s lifetime. Both argue for a denser mesh; routing resource availability argues for a sparser one. On this design the power consumption is low enough that the default OpenLane PDN configuration is comfortable, but on a larger design IR-drop analysis is a signoff step in its own right.

7.6 Placement

Purpose. Assign every standard cell a physical location within the core.

Placement proceeds in two phases:

Global placement (RePLace) treats placement as a continuous optimisation problem. Cells are modelled as movable objects; wirelength is approximated by a smooth differentiable function (log-sum-exp of half-perimeter wirelength); density is modelled as an electrostatic field where cell overlap generates a repulsive force. Gradient descent on the combined objective produces a distribution that is near-optimal for wirelength but in which cells overlap and sit off-grid.

Detailed placement (OpenDP) legalises that result: every cell is snapped to a valid site on a standard-cell row, all overlaps are removed, and the perturbation from the global positions is minimised so that the wirelength quality is preserved.

Table 7.4 — Placement configuration parameters

Parameter	Effect
PL_TARGET_DENSITY	Density target for global placement; typically set slightly above FP_CORE_UTIL
PL_BASIC_PLACEMENT	Bypass timing-driven placement (debug only)
PL_TIME_DRIVEN	Weight critical-path nets more heavily in the objective
PL_RESIZER_*	Post-placement timing repair: buffer insertion and cell resizing

Two additional operations occur around placement and are easy to overlook:

- **Tap cell insertion** places well-tap cells at a regular pitch to tie the n-well and p-substrate to supply, preventing latch-up. This is a foundry rule, not an optimisation.
- **Decap and filler cell insertion** fills the gaps between placed cells. Fillers maintain well continuity and satisfy density rules; decap cells add local supply decoupling capacitance.

Figure 7.5 placeholder — [Insert Figure 7.5: Global placement and density map] **Figure 7.6 placeholder** — [Insert Figure 7.6: Legalised detailed placement]

7.7 Clock Tree Synthesis

Purpose. Build a buffered distribution network that delivers the clock from its source to every sequential element with balanced arrival times.

Before CTS, the clock is an ideal net in the timing model: it reaches every flip-flop instantaneously. That assumption is what makes pre-CTS timing reports optimistic and, on their own, not trustworthy. CTS replaces the ideal net with real buffers and real wires, after which the timing picture becomes meaningful.

The objectives:

Objective	Definition	Why it matters
Skew	Max – min clock arrival across sinks	Directly consumes setup and hold margin
Latency (insertion delay)	Source-to-sink delay	Affects I/O timing and on-chip variation sensitivity
Transition time	Clock edge rate at each sink	Slow edges increase jitter sensitivity and short-circuit power
Power	Clock tree switching power	The clock switches every cycle on every branch; typically 20–40 % of total dynamic power

Table 7.5 — CTS configuration parameters

Parameter	Effect
CTS_TARGET_SKEW	Skew the tool aims to achieve
CTS_ROOT_BUFFER	Buffer cell used at the tree root
CTS_CLK_BUFFER_LIST	Candidate buffers for internal levels
CTS_SINK_CLUSTERING_SIZE	Sinks grouped per leaf-level buffer

TritonCTS clusters sinks geometrically, builds a buffer tree over the clusters, and then balances branch delays by buffer sizing and wire snaking. Hold violations frequently *appear* after CTS: with an ideal clock, launch and capture were simultaneous; with a real tree, a short data path between two flip-

flops whose clock arrives with a small skew can violate hold. These are repaired by inserting delay buffers on the data path, which is why cell count typically grows at this stage.

Figure 7.7 placeholder — [Insert Figure 7.7: Clock tree structure] Source: runs/<run_tag>/reports/cts/

7.8 Routing

Purpose. Realise every logical net as physical metal on the available layers, connecting all its pins while obeying every design rule.

Global routing (FastRoute) partitions the core into a coarse grid of global routing cells and determines, for each net, the sequence of cells it passes through and the layers it uses — without committing to exact wire geometry. Its purpose is to distribute demand across the available capacity. The output of this stage is a congestion map, and reading it is the single most useful diagnostic in physical design: a localised region of demand above capacity predicts exactly where detailed routing will fail.

Detailed routing (TritonRoute) commits every net to specific tracks, exact coordinates and via placements, honouring the full design rule set: minimum width, minimum spacing, via enclosure, minimum area, end-of-line spacing, and parallel-run-length spacing rules.

Table 7.6 — Routing configuration parameters

Parameter	Effect
RT_MIN_LAYER / RT_MAX_LAYER	Layer range available to the signal router
GLB_RT_ADJUSTMENT	Derating applied to global routing capacity
ROUTING_CORES	Parallelism for detailed routing
DRT_OPT_ITERS	Detailed routing optimisation iteration limit

The SKY130 metal stack, from the router’s perspective:

Layer	Typical use	Direction
li1	Local interconnect, cell-internal, pin access	—
met1	Short local nets, standard-cell power rails	Horizontal
met2	Local routing	Vertical
met3	Intermediate routing	Horizontal
met4	Longer routing, PDN straps	Vertical
met5	Global routing, PDN straps	Horizontal

Alternating preferred directions per layer is what makes routing tractable: any two points can be connected by a staircase of horizontal and vertical segments with via transitions, and nets on adjacent layers rarely run parallel for long distances.

Figure 7.8 placeholder — [Insert Figure 7.8: Global routing congestion map] **Figure 7.9 placeholder** — [Insert Figure 7.9: Detailed routing, all layers] **Figure 7.10 placeholder** — [Insert Figure 7.10: Metal utilisation by layer]

7.9 Parasitic Extraction and Static Timing Analysis

Extraction (OpenRCX) computes the resistance and capacitance of every routed wire from its geometry and the process stack, producing a SPEF file. Before extraction, wire delay is estimated; after it, wire delay is calculated from actual geometry. This is why post-route STA is the only timing result that should be quoted as a design’s achieved performance.

Static timing analysis (OpenSTA) enumerates every timing path in the design and computes, for each, whether the data arrives within its required window. It is exhaustive and vector-independent, which is why it — not simulation — is the signoff mechanism for timing.

The two checks:

Check	Condition	Failure mode	Fix
Setup	Data must arrive before the capture edge minus setup time	Too slow	Faster/larger cells, shorter paths, longer period, pipelining
Hold	Data must not change until after capture edge plus hold time	Too fast	Insert delay buffers

The asymmetry between them is important. A setup violation can be fixed by slowing the clock — the chip works, just at a lower frequency. A hold violation cannot be fixed by any clock frequency, because it is a race between two paths independent of the period. A hold violation in silicon is a dead chip. This is why hold is fixed aggressively and conservatively in the flow, at the cost of inserted buffers and the area and power they consume.

Key reported quantities:

Metric	Meaning
WNS	Worst negative slack — the single most violating path
TNS	Total negative slack — sum over all violating paths
Number of violating endpoints	Breadth of the problem

WNS and TNS together diagnose the problem: a large WNS with a small TNS is one bad path, usually fixable locally. A modest WNS with a large TNS is a systemic problem — the frequency target is too aggressive, or the floorplan is wrong.

7.10 Physical Verification: DRC

Purpose. Verify that the layout obeys every geometric rule the foundry requires for manufacturability.

Rules are derived from lithography, etch and deposition limits, and cover:

Rule class	Constraint
Minimum width	Narrower features do not print reliably
Minimum spacing	Closer features bridge
Minimum area	Small isolated shapes lift or fail to form
Enclosure	A via must be fully covered by the metal above and below
Density	Metal density must fall within a window for CMP planarity
Antenna	Limits on the conductor area attached to a gate during processing

In OpenLane, DRC is run by Magic against the SKY130 rule deck and cross-checked by KLayout, which uses an independently written deck. Running both is genuinely valuable — a violation that appears in one and not the other usually indicates a deck interpretation difference rather than a real layout problem, and knowing which is which saves considerable time.

Figure 9.5 placeholder — [Insert Figure 9.5: DRC report summary] Source: runs/<run_tag>/reports/signoff/

7.11 Physical Verification: LVS

Purpose. Prove that the devices and connectivity in the layout match the gate-level netlist.

The procedure: Magic extracts a SPICE netlist from the layout geometry by recognising transistor structures and tracing connectivity; Netgen compares that extracted netlist against the synthesised netlist, performing graph isomorphism with device parameter checking.

Typical mismatch causes and what they indicate:

Symptom	Usual cause
Device count mismatch	Missing fill or tap cells; extraction problem
Net count mismatch	Shorted nets in layout, or an open
Property mismatch	Transistor width/length differ from the library cell
Unmatched instances	Naming or hierarchy mismatch between views

DRC and LVS together are the tape-out gate. DRC says the layout can be manufactured; LVS says that what is manufactured is the intended circuit. Neither implies the other, and a design passing only one is not signed off.

Figure 9.6 placeholder — [Insert Figure 9.6: LVS report summary]

7.12 Antenna and Density Checks

Antenna effect. During plasma etch, a long metal segment connected to a gate but not yet connected to a diffusion acts as a charge collector; accumulated charge can damage the thin gate oxide. The rule limits the ratio of connected metal area to gate area at each processing step. Fixes are layer jumping (route up to a higher layer and back, breaking the long segment at the stage in question) or inserting an antenna diode to provide a discharge path.

Density. Chemical-mechanical polishing planarises each layer, and the removal rate depends on local pattern density. Regions that are too sparse dish; regions too dense polish unevenly. Foundries therefore specify minimum and maximum density per layer within a sliding window, satisfied by inserting metal fill in sparse regions.

Both checks are easy to overlook because they have nothing to do with logical correctness — and both will stop a tape-out.

7.13 GDSII Stream-Out

The final stage merges the routed design with the standard-cell layouts from the PDK and streams the result to a GDSII file: a hierarchical binary database of polygons, each tagged with a layer and datatype number, that the mask shop consumes directly.

The output is `runs/<run_tag>/results/final/gds/sdram_controller.gds`.

Chapter 8 — Tools and Environment

Table 8.1 — Tool inventory, version and role

Tool	Version	Category	Role in this project
Verilog-2001	IEEE 1364-2001	Language	Design entry
Xilinx Vivado	<version>	Simulation / elaboration	RTL elaboration, schematic view, behavioural simulation
OpenLane	<version>	Flow orchestration	Automated RTL-to-GDSII
Yosys	<version>	Synthesis	RTL to generic logic
ABC	bundled	Technology mapping	Generic logic to SKY130 cells
OpenROAD	<version>	Physical design	Floorplan, PDN, placement, CTS, routing
OpenSTA	<version>	Timing	Static timing analysis
OpenRCX	bundled	Extraction	RC parasitic extraction to SPEF
Magic	<version>	Layout / verification	DRC, layout extraction for LVS, GDS view
Netgen	<version>	Verification	LVS netlist comparison
KLayout	<version>	Layout / verification	Independent DRC, GDS viewing and screenshots
SKY130 PDK	sky130A	Technology	Cell libraries, rule decks, tech LEF
Docker	<version>	Environment	Reproducible tool container
GTKWave	<version>	Waveform	VCD viewing outside Vivado
Git / GitHub	—	Version control	Source management, release

Notes on individual tools

Yosys. The open-source synthesis front end. Its value in a learning context is that its intermediate representation is inspectable at every pass — [show](#) renders the design graph after any transformation, which makes the effect of each optimisation directly visible rather than a black-box result.

OpenROAD. The physical design engine, integrating RePlace (global placement), OpenDP (legalisation), TritonCTS, FastRoute and TritonRoute behind a single Tcl interface. Each component corresponds to an identifiable stage in a commercial flow.

Magic. A layout editor dating to Berkeley in the 1980s, still maintained and still capable. In this flow it performs DRC against the SKY130 deck and extracts the layout netlist for LVS.

KLayout. A fast, scriptable layout viewer and DRC engine. Used here both for independent DRC cross-checking and for producing publication-quality layout images.

Docker. OpenLane depends on a large set of tools at specific versions. Container distribution removes the entire class of build-environment problems and makes a run reproducible on another machine — which is the difference between a result and an anecdote.

Vivado. Used for RTL elaboration, the schematic view of Figure 4.4, and behavioural simulation. Its role in this project was as a design-entry and verification environment, not as an implementation target.

SKY130 PDK. SkyWater’s 130 nm process, released as open source in collaboration with Google. The significance is difficult to overstate: it is the first freely licensed process design kit with a real manufacturing path, and it is the reason this project could target a real technology rather than an academic abstraction.

Figure 8.1 placeholder — [Insert Figure 8.1: Tool flow and data handoff]

8.1 FPGA Prototyping Environment

An FPGA toolchain was used alongside the ASIC flow for RTL elaboration and simulation. This is standard industrial practice — RTL is validated on a fast-turnaround target before committing to the physical flow — but the two results must be reported separately, because FPGA implementation data (LUT counts, slice placement, device floorplan) describes a different physical realisation from the ASIC one and says nothing about the ASIC result.

Item	Detail
Tool	Xilinx Vivado
Target device	<part number> — see Figure 8.2
Purpose	RTL elaboration, schematic inspection, behavioural simulation
Not used for	Any result reported in Chapter 9

Figure 8.2 placeholder — [Insert Figure 8.2: FPGA project part selection] Source file: [images/Screenshot_2026-07-21_215625.png](#)

Figure 8.3 placeholder — [Insert Figure 8.3: FPGA device view – clock regions] Source file: [images/Screenshot_2026-07-27_203642.png](#)
Caption: Vivado device view showing FPGA clock regions X0Y0 through X1Y1. **This is an FPGA fabric view, not an ASIC floorplan.**

Figure 8.4 placeholder — [Insert Figure 8.4: FPGA device view – placed logic] Source file: [images/Screenshot_2026-07-27_203720.png](#)
Caption: Vivado device view showing configurable logic blocks and slices with a flip-flop BEL selected. **This is FPGA fabric occupancy, not ASIC standard-cell placement.**

Chapter 9 — Results

How to complete this chapter. Every numeric field marked `<value>` must be populated from the OpenLane run directory. The source path for each is given beneath its table. No value in this chapter should be estimated, carried over from a different run, or filled in from a similar design — a results table with a number that does not match the reports it cites is the fastest way to lose a technical reviewer’s confidence.

9.1 RTL Schematic

Description. The elaborated schematic confirms the structural hierarchy prior to optimisation.

Observation. All four submodules are instantiated and connected as specified in Chapter 4. No unconnected ports, no inferred top-level glue logic.

Result. RTL hierarchy matches the architecture specification.

See Figure 4.4.

9.2 Functional Simulation

Description. Behavioural simulation of the integrated controller against directed stimulus.

Observation. Address decoding matched hand calculation at all five test points (Table 6.2). Command sequences matched the expected sequence for each access class. Address latching behaved as designed.

Result. Functional intent confirmed for the implemented feature set, subject to the verification gaps recorded in Section 6.7.

See Figures 6.2, 6.3, 6.4

9.3 Synthesis Results

Table 9.1 — Synthesis results summary

Metric	Value	Source
Top module	sdram_controller	—
Target library	sky130_fd_sc_hd	—
Total cell count	<value>	reports/synthesis/1-synthesis.stat.rpt
Sequential cells (flip-flops)	<value>	same
Combinational cells	<value>	same
Buffers / inverters	<value>	same
Estimated combinational area	<value> μm²	same
Estimated sequential area	<value> μm²	same
Estimated total cell area	<value> μm²	same
Number of nets	<value>	same
Chip area (post-synthesis estimate)	<value> μm²	same

Engineering analysis prompts. When populating this table, comment on: whether the flip-flop count matches the architectural expectation (roughly 52 for the bank manager state, plus FSM state bits, plus the address latches, plus counters); whether any unexpected cell types appear; and whether the buffer count is proportionate to the design size.

9.4 Area Utilisation

Table 9.2 — Area utilisation summary

Metric	Value	Source
Die width × height	<value> × <value> μm	reports/ or final DEF header
Die area	<value> μm²	derived
Core area	<value> μm²	DEF DIEAREA / core box
Standard cell area	<value> μm²	reports/
Core utilisation (achieved)	<value> %	reports/
Standard cell rows	<value>	DEF
Filler cells inserted	<value>	reports/
Tap cells inserted	<value>	reports/
Decap cells inserted	<value>	reports/

9.5 Floorplan, Placement and CTS

For each of these stages, capture a screenshot from the OpenROAD GUI or by opening the stage DEF in KLayout, and complete the following:

Stage	Figure	DEF source
Floorplan	7.3	results/floorplan/sdram_controller.def
Power plan	7.4	same, after pdngen
Global placement	7.5	results/placement/
Detailed placement	7.6	results/placement/sdram_controller.def
Clock tree	7.7	results/cts/sdram_controller.def
Global routing congestion	7.8	reports/routing/
Detailed routing	7.9	results/routing/sdram_controller.def

CTS quality metrics to report:

Metric	Value	Source
Clock tree buffer count	<value>	reports/cts/
Clock tree depth (levels)	<value>	same
Achieved clock skew	<value> ps	STA clock report
Clock insertion delay	<value> ps	same
Number of clock sinks	<value>	same

9.6 Static Timing Analysis

Table 9.3 — Static timing analysis summary (post-route, extracted parasitics)

Metric	Setup	Hold	Source
Worst negative slack (WNS)	<value> ns	<value> ns	reports/signoff/*sta*
Total negative slack (TNS)	<value> ns	<value> ns	same
Violating endpoints	<value>	<value>	same
Achieved maximum frequency	<value> MHz	—	derived: $1 / (T - WNS)$
Target frequency	<value> MHz	—	config.tcl
Critical path start point	<instance/pin>		timing report
Critical path end point	<instance/pin>		timing report
Critical path logic depth	<value> levels		timing report

Engineering analysis prompts. Identify where the critical path runs. In this architecture, the likely candidates are the combinational query path (address decoder → command generator → bank manager query → row comparison → next-state logic), and the delay counter comparison feeding the FSM. If the critical path is the former, the fix described in Section 4.8 — registering the bank query response and adding a DECODE cycle — is the architectural response.

Figure 9.7 placeholder — [Insert Figure 9.7: Post-route timing histogram]

9.7 Power Estimation

Table 9.4 — Power estimation summary

Component	Value	Percentage	Source
Internal power	<value> μ W	<%>	reports/signoff/*power*
Switching power	<value> μ W	<%>	same
Leakage power	<value> μ W	<%>	same
Total	<value> μ W	100 %	same
Clock network power	<value> μ W	<%>	same
Combinational logic power	<value> μ W	<%>	same
Sequential logic power	<value> μ W	<%>	same

Caveat that must accompany these numbers. OpenLane’s power estimate is based on default switching activity assumptions, not on activity extracted from simulation. It is a rough figure suitable for relative comparison between runs, not an absolute prediction of silicon power. An accurate estimate requires back-annotating a switching activity file (SAIF or VCD) from a representative simulation workload. State this explicitly wherever these numbers are quoted.

Figure 9.8 placeholder — [Insert Figure 9.8: Power breakdown]

9.8 Physical Verification Signoff

Table 9.5 — Physical verification signoff summary

Check	Tool	Result	Violations	Source
DRC	Magic	<PASS/FAIL>	<count>	reports/signoff/
DRC (independent)	KLayout	<PASS/FAIL>	<count>	same
LVS	Netgen	<PASS/FAIL>	<count>	same
Antenna	Magic	<PASS/FAIL>	<count>	same
Connectivity / open nets	OpenROAD	<PASS/FAIL>	<count>	same

9.9 Final GDSII Layout

Description. The completed layout database, containing the placed and routed standard cells, the power distribution network, the clock tree, all signal routing across the metal stack, and the boundary pin geometry.

Observation. The die shows the expected structure: a dense, uniformly filled core of standard-cell rows with routing across all metal layers, bounded by a power ring, with I/O pin labels distributed around the periphery. Uniform core density with no visible voids indicates that placement legalisation and filler insertion completed correctly. The absence of visible routing congestion hotspots is consistent with a successful detailed routing run.

Result. GDSII generated. Signoff status per Table 9.5.

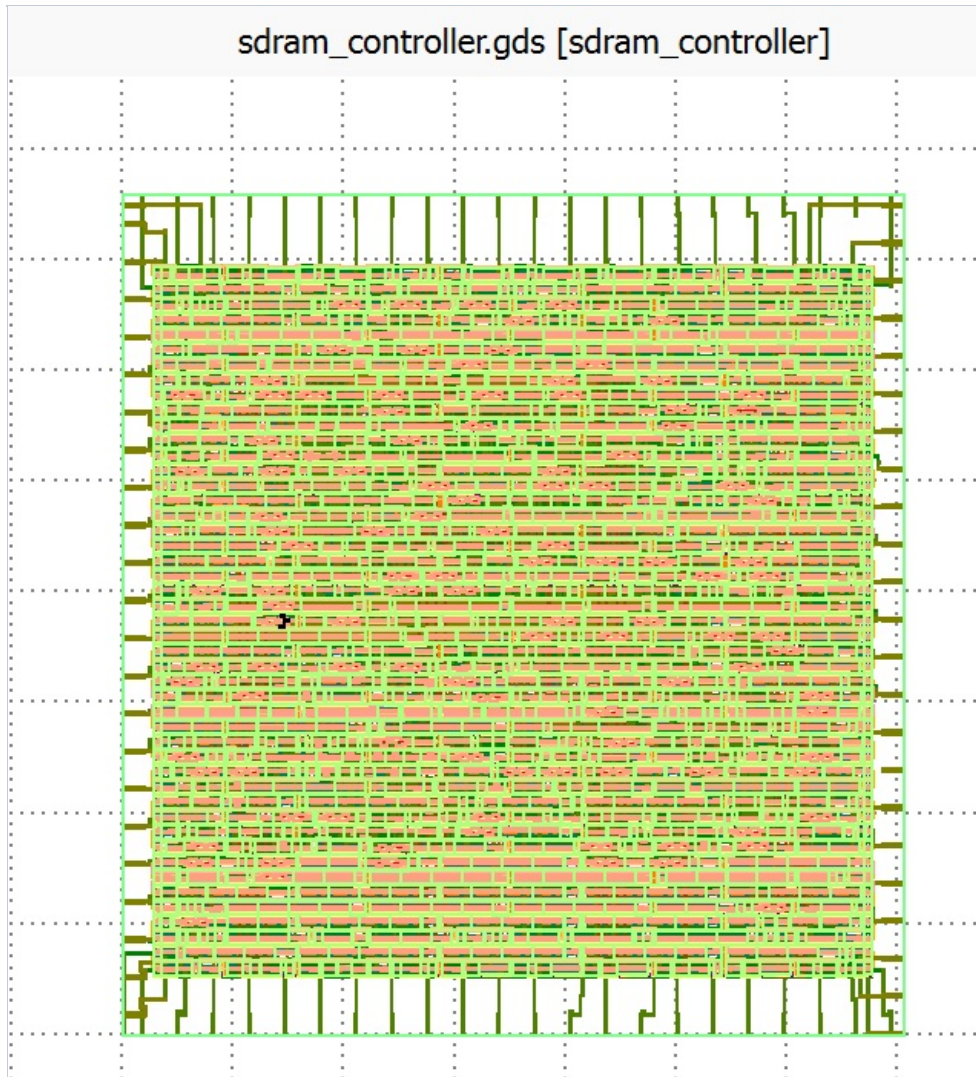


Figure 9.1 — Final GDSII layout of `sdram_controller`, label rendering disabled. The uniformly filled core of standard-cell rows is bounded by the power ring, with the peripheral pin structures visible around the boundary. The regular horizontal banding is the standard-cell row structure with its met1 power rails; the finer texture above it is signal routing on the upper metal layers. The absence of voids in the core indicates that placement legalisation and filler insertion completed correctly, and the absence of visible congestion hotspots is consistent with a clean detailed-routing run.

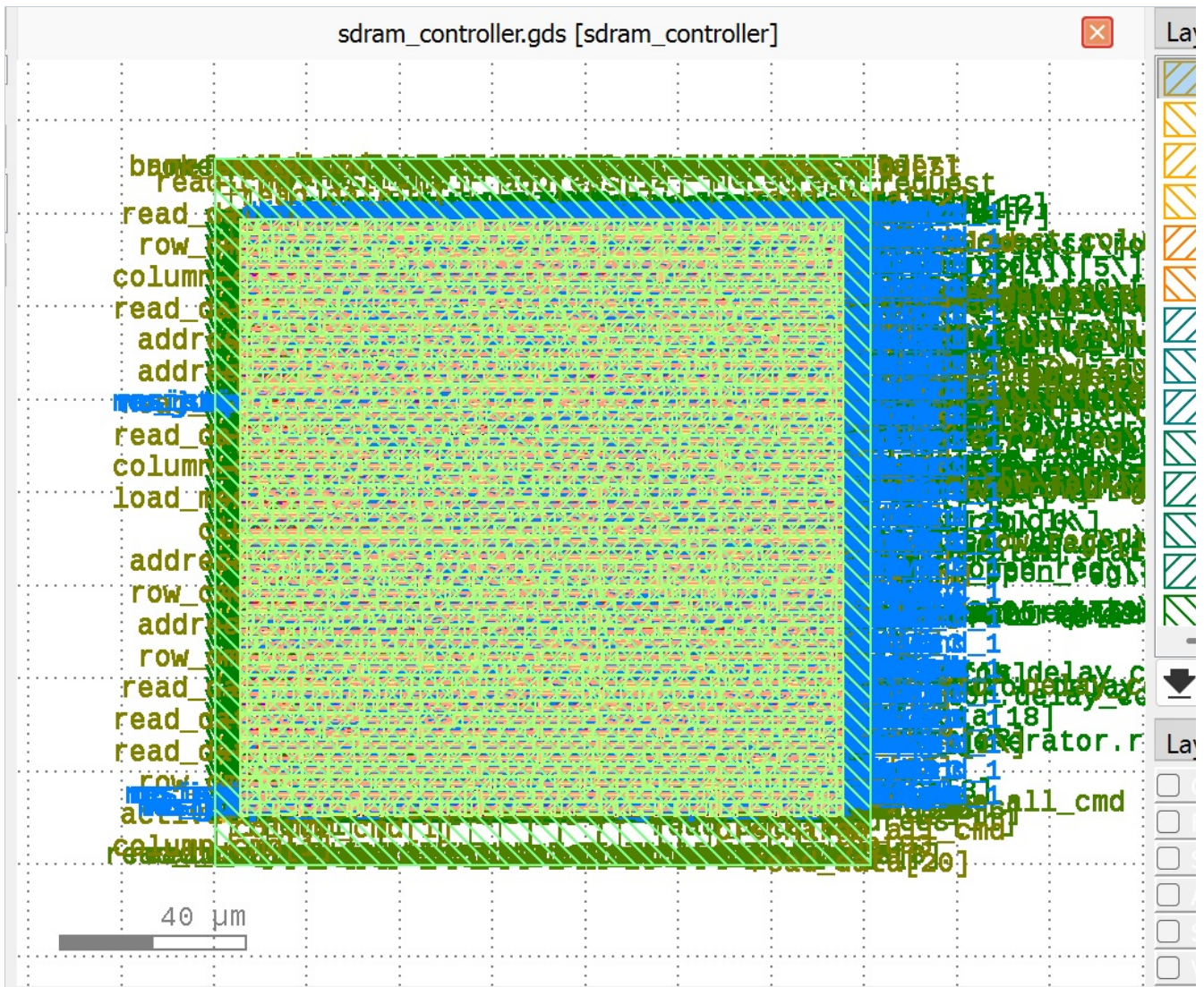


Figure 9.2 — The same GDSII database viewed in Magic with net-name label rendering enabled. Pin labels for the top-level ports — `read_request`, `read_data`, `row_cmd`, `column_cmd`, `address`, `load_mode_cmd` and others — are visible around the boundary, confirming that the boundary pin set matches the top-level port list of Table 4.1. Scale bar: 40 μm .

Chapter 10 — Design Challenges and Resolutions

Table 10.1 — Issue log

ID	Area	Symptom	Root cause	Resolution
I-01	RTL	Bank state not cleared by reset under some stimulus	Reset coded as a plain <code>if</code> followed by a second plain <code>if</code> rather than an <code>if / else if</code> chain	Restructured to a strict priority chain
I-02	RTL	Bank manager reported the wrong open row	12-bit vector indexed by a 2-bit bank index — a single-bit write	Redeclared as a 2-D array <code>reg [11:0] active_row_reg [3:0]</code>
I-03	RTL	Width mismatch warnings on the bank-open signal	Scalar output port versus 4-bit internal state array	Output driven as <code>bank_open_reg[query_bank]</code>
I-04	RTL	<code>precharge_all_cmd</code> had no effect on bank state	Input declared but not handled in the update logic	Added the all-banks clear branch
I-05	Verification	Simulation runtime dominated by the power-up wait	100 μ s wait is 10,000 cycles	Parameter override from the testbench
I-06	Verification	Decoded fields appeared wrong in the waveform	Misreading strobe-qualified buses as continuously valid	Understood and documented the protocol (Section 4.2)
I-07	Flow	OpenLane container failed to start / mount	Docker volume, permission and path configuration	Corrected mount paths and user mapping
I-08	Flow	Detailed routing failed to converge	Core utilisation too high; insufficient routing resource	Reduced <code>FP_CORE_UTIL</code> , re-ran from floorplan
I-09	Flow	Hold violations appeared after CTS	Ideal-clock assumption replaced by a real tree	Allowed the resizer to insert hold buffers
I-10	Signoff	Antenna violations on long nets	Metal segment area attached to a gate exceeded the rule	Layer jumping / diode insertion via flow settings

10.1 Discussion — Flow and Environment

I-07: OpenLane and Docker bring-up. The first substantial obstacle in a project like this is usually not the design. OpenLane runs inside a container, and getting the container to see the design directory, write results back with usable ownership, and locate the PDK correctly involves several interacting configuration items: the PDK root environment variable, the design directory mount, and the user ID mapping between host and container. The characteristic symptom is a flow that starts, runs a stage, and fails with a file-not-found or permission error on a path that exists.

The productive approach is to verify the environment incrementally rather than debug it through a failing flow run: confirm the container starts, confirm the PDK path resolves inside it, run the flow on one of the bundled example designs, and only then substitute the design under development. A flow that fails on `spm` — the shipped example — is an environment problem; a flow that succeeds on `spm` and fails on this design is a design or configuration problem. Separating those two cases early saves considerable time.

I-08: Routing convergence. The most common physical-design failure for a first project. Detailed routing iterates, reports a decreasing but non-zero violation count, and eventually terminates without a clean result. The instinct is to increase the router’s iteration limit; this rarely helps, because the problem is not effort but resource.

The correct diagnostic sequence is: look at the global routing congestion map, identify whether congestion is uniform or localised, and respond accordingly. Uniform congestion means the core is too small — reduce `FP_CORE_UTIL` and re-run from floorplan. Localised congestion means a specific region is over-subscribed, usually because a high-fanout net or a cluster of tightly connected cells was placed together; the response is a placement adjustment or a region constraint. Raising the layer range available to the router can also help if the upper layers are under-used.

The general lesson is worth stating explicitly, because it is the main thing a first physical-design project teaches: **routing failures are usually floorplan problems**. By the time the router fails, the decision that caused the failure was made several stages earlier.

I-09: Post-CTS hold violations. These are expected, not anomalous. Before CTS the clock is ideal and launch and capture edges are simultaneous, so short data paths pass hold trivially. After CTS, real skew exists, and any path shorter than the skew between its two endpoints violates hold. The flow repairs these by inserting delay buffers. A large number of inserted hold buffers is a signal that clock skew is higher than it should be, which points back at CTS configuration — but a moderate number is normal and should not be treated as a problem.

I-10: Antenna violations. These appear only at signoff and have nothing to do with logical correctness, which makes them a surprise the first time. The fix is largely automatic through flow settings (layer jumping or diode insertion), but understanding the underlying mechanism — charge accumulation on a metal segment during plasma processing damaging a gate oxide — is what distinguishes knowing the fix from knowing the problem.

10.2 Discussion — RTL Defects in the Bank Manager

Four of the ten logged issues (I-01 to I-04) were in one module, which is not coincidental: the bank manager is the only module in the design holding indexed array state, and indexed array state in Verilog is where width and indexing errors concentrate. This section documents them in detail because they are instructive and because each is a defect a reviewer would expect to see identified rather than quietly fixed.

Figure 10.1 placeholder — [Insert Figure 10.1: Bank manager source during debug] Source file: `images/Screenshot_2026-07-26_185411.png` Caption: Editor view of the bank manager update logic during debug, showing the defects described below flagged by the linter.

I-01 — Reset without priority. The update logic was structured as:

```
if (reset) begin
    bank_open_reg[0] <= 0;
    ...
end
if (activate_cmd) begin          // <-- should be: else if
    ...
end
if (precharge_cmd) begin        // <-- should be: else if
    ...
end
```

Because the second and third conditions are independent `if` statements rather than continuations of the first, an `ACTIVATE` asserted in the same cycle as reset executes *after* the reset branch and overwrites it. The design is left in a state where reset does not reliably reset. This passes any test in which no command is ever asserted during reset — which is most tests, written naturally. It is the reason TC-12 (reset during an active transaction) belongs in the test plan.

The fix is a strict priority chain: `if (reset) ... else if (precharge_all_cmd) ... else if (activate_cmd) ... else if (precharge_cmd) ...`.

I-02 — Vector indexed as an array. The row-tracking assignment was written as:

```
active_row[bank_cmd] <= row_cmd;
```

where `active_row` is a 12-bit vector and `bank_cmd` is a 2-bit index. This does not store `row_cmd` in a per-bank slot. It selects a *single bit* of `active_row` — bit 0, 1, 2 or 3 depending on the bank — and assigns it the least significant bit of `row_cmd`. The other eleven bits of the row value are silently discarded.

This is a particularly dangerous class of error because it is legal Verilog. It elaborates, it synthesises, and it produces a bank manager whose reported open row is meaningless — so the row-hit comparison in the command generator produces essentially random results, and the controller occasionally skips an `ACTIVATE` it needed or issues one it did not.

The correct declaration separates the array dimension from the word width:

```
reg      bank_open_reg [3:0];      // 4 banks × 1 bit
reg [11:0] active_row_reg [3:0];   // 4 banks × 12 bits

// update
active_row_reg[bank_cmd] <= row_cmd;

// query
assign bank_open = bank_open_reg[query_bank];
assign active_row = active_row_reg[query_bank];
```

I-03 — Inconsistent naming between reset and update branches. The reset branch assigned `bank_open_reg[0..3]` while the command branches assigned `bank_open[...]`. If `bank_open` is the output port and `bank_open_reg` the internal storage, these are two different objects, and the module ends up with a reset that clears storage the rest of the logic never writes. The `_reg` suffix convention of Table 5.2 exists precisely to make this distinction visible, and it only works if applied consistently in every branch.

I-04 — An input with no effect. `precharge_all_cmd` was present in the port list and connected at the top level but had no branch in the update logic. The initialization unit's `PRECHARGE ALL` therefore left the bank manager's state untouched. Because the state is already all-zero at that point in the sequence, the omission has no observable effect during initialization — it only matters if `PRECHARGE ALL` is ever issued during normal operation, which is exactly what the refresh scheduler proposed in Chapter 11 will do.

This is a latent defect: correct today, incorrect the moment a planned feature is added. Latent defects of this kind are what lint tools that flag unused input ports are for.

10.3 Discussion — Verification

I-05: Simulation runtime. The 100 μ s power-up wait is 10,000 cycles at 100 MHz. A simulation that must traverse it before any interesting behaviour occurs is slow to run and painful to inspect. The resolution was a parameter override from the testbench, and Section 5.7 explains why parameter override rather than conditional compilation is the safe form of this shortcut.

I-06: Misreading the strobe-qualified bus. During bring-up, `row_cmd` and `column_cmd` appeared to hold values inconsistent with the `address` input, which was initially read as a decoder fault. It was not: the buses are only valid during their command strobe, and the values held are the *latched* decode of the accepted request rather than a live decode of whatever is currently on `address`. Figure 6.3 shows exactly this.

The general lesson is that a protocol's validity rules are part of its specification, and a waveform cannot be interpreted without them. This is also the strongest single argument for assertions: the property “`row_cmd` is stable and equal to the latched row whenever `activate_cmd` is asserted” is one line of SVA, and it converts an interpretation question into an automatic check.

Chapter 11 — Future Work

The extensions below are ordered by dependency, not by appeal. Several of them are prerequisites for others, and implementing them out of order forces rework — the sequencing note against each explains why.

Table 11.1 — Future work items with dependency ordering

#	Item	Depends on	Effort	Rationale
1	Close the read data path	—	S	Section 6.7 gap 2 — required for the design to be a complete controller
2	Timing-accurate SDRAM model in the testbench	1	M	Section 6.7 gap 1 — required before any protocol-compliance claim
3	Write support	1	M	Refresh arbitration must be designed against a design that has writes
4	Auto-refresh controller and scheduler	3	M	Data retention; must arbitrate against pending writes
5	SVA assertions	2	S	Cheapest while the timing logic is fresh
6	Self-checking scoreboard	2	M	Enables automated regression
7	Request queue / FIFO	3, 4	M	Prerequisite for outstanding requests and for AXI4
8	Multiple outstanding requests	7	L	The largest single performance gain available
9	Burst read and write	4, 7	M	Burst interruption by refresh is hard to retrofit
10	AXI4-Lite interface	7	M	Natural first standard interface
11	AXI4 (full) interface	9, 10	L	Needs bursts and IDs underneath
12	Functional coverage	6	M	Quantifies verification completeness
13	Clock gating and low-power optimisation	—	M	Independent work package
14	Power-down and self-refresh states	4	M	Needs the refresh scheduler
15	ECC (SECCDED)	3	L	Requires a write path to store check bits
16	DDR signalling support	9	XL	A different class of project — analogue PHY involved
17	UVM verification environment	6, 12	XL	Industry-standard methodology

11.1 Close the Read Data Path

The immediate priority. Add an SDRAM data input port, connect the CAS-latency-delayed capture register to it, and drive `read_data` from that register. Until this is done the design is a command sequencer rather than a memory controller, and the report should say so — which is why Section 6.7 states it plainly.

11.2 Write Support

Writes are architecturally similar to reads but differ in three ways that matter:

- Write data accompanies the WRITE command rather than following it after a latency.
- `tWR` (write recovery) must elapse between the last write data and a PRECHARGE, because the data must propagate from the sense amplifiers into the cells.
- Byte enables / data masks become meaningful, which is where `address[1:0]` finally earns its place.

The FSM gains a parallel branch from DECODE; the bank manager is unaffected.

Sequencing note. Write support must precede the refresh controller. Refresh arbitration has to decide what to do when a refresh becomes due while a write is in flight; designing that arbiter against a read-only datapath and then adding writes means rewriting it.

11.3 Auto-Refresh Controller and Scheduler

A tREFI counter generates a refresh request; an arbiter decides when to service it. The minimum viable version issues PRECHARGE ALL followed by AUTO REFRESH as soon as the counter expires and the current transaction completes. A better version exploits the specification's allowance for deferring up to eight refreshes, issuing them opportunistically during idle periods and catching up when necessary.

Sequencing note. Refresh must precede bursting. Once refresh can pre-empt a transaction, a burst in flight must either be allowed to complete or be cleanly interrupted, and retrofitting that logic into a burst FSM designed without it is substantially harder than designing the burst FSM with refresh already present.

11.4 Assertion-Based Verification

Properties worth encoding as SVA, in rough priority order:

Property
No two command strobes are asserted in the same cycle
<code>read_cmd</code> is never asserted unless the target bank has a row open
At least tRCD cycles separate <code>activate_cmd</code> from <code>read_cmd</code> to the same bank
At least tRP cycles separate <code>precharge_cmd</code> from <code>activate_cmd</code> to the same bank
At least tRAS cycles separate <code>activate_cmd</code> from <code>precharge_cmd</code> to the same bank
<code>data_valid</code> is asserted exactly CL cycles after <code>read_cmd</code>
No command is issued while <code>init_done</code> is low, other than by the initialization unit
<code>bank_cmd</code> , <code>row_cmd</code> and <code>column_cmd</code> are stable whenever their strobe is asserted
A bank never has two rows open simultaneously
<code>req_ack</code> is asserted only when <code>ready</code> was asserted in the same cycle

The tRAS property is the one that closes the gap identified in Sections 4.7 and 6.7 — it is a single line and it proves something the directed tests do not.

11.5 Low-Power Techniques

Technique	Applicability
Clock gating	High — most of the logic is idle most cycles; largest single win
Multi-Vt cell selection	Moderate — high-Vt on non-critical paths reduces leakage
Operand isolation	Low — little datapath logic to isolate
Power gating	Low for this size; relevant if a power-down state is added
Voltage scaling	Out of scope — requires multi-corner characterisation

Clock gating should be the first step and would need a re-run of CTS with the gated tree.

11.6 Standard Bus Interface

The existing `ready` / `req_ack` / `data_valid` handshake maps directly onto AXI4-Lite's channel structure:

Controller signal	AXI4-Lite equivalent
<code>read_request</code>	<code>ARVALID</code>
<code>ready</code>	<code>ARREADY</code>
<code>address</code>	<code>ARADDR</code>
<code>read_data</code>	<code>RDATA</code>
<code>data_valid</code>	<code>RVALID</code>
(not present)	<code>RREADY</code> — host backpressure, must be added
(not present)	<code>RRESP</code> — response code

The missing `RREADY` is significant: AXI permits the host to stall the response, which requires the controller to hold data rather than pulse it. This is another reason the FIFO (item 7) is a prerequisite.

11.7 Physical Design Extensions

Extension	Value
Multi-corner multi-mode STA	Verify across process, voltage and temperature corners rather than one
Activity-annotated power analysis	Replace default-activity estimates with SAIF-based figures
IR drop analysis	Verify the PDN under realistic current demand
DFT scan insertion	Stitch a scan chain; report scan coverage
Floorplan exploration	Sweep utilisation and aspect ratio; plot area against achieved frequency
Manual macro placement	Relevant once a memory macro is introduced

The floorplan exploration item deserves emphasis. Running the flow once produces a result; running it across a parameter sweep and plotting the trade-off surface produces *understanding*, and a plot of achieved frequency against core utilisation is a far more compelling portfolio artefact than a single number.

Chapter 12 — Conclusion

This project took an SDRAM controller from a behavioural Verilog description to a signed-off GDSII layout in a 130 nm process, traversing every stage of the digital ASIC flow without omission.

What was built. A four-module synchronous controller comprising an address decoder mapping a 24-bit flat address onto a 4-bank $\times$ 4096-row $\times$ 256-column geometry, a bank manager tracking open-row state per bank, an initialization unit executing the mandated power-up sequence, and a command generator containing the control FSM and timing enforcement logic. The address map was verified against simulation at five points; the command sequences for row-hit, row-miss and row-conflict accesses were confirmed; and the design was carried through synthesis, floorplanning, power planning, placement, clock tree synthesis, routing, extraction, static timing analysis, and DRC and LVS signoff to a GDSII database.

What was learned. Several things that are difficult to acquire any other way:

- *That the flow is a sequence of constrained transformations, each with its own failure mode.* Encountering a routing failure that turns out to be a floorplan problem teaches the coupling between stages in a way that reading about it does not.
- *That timing closure is a design activity, not a back-end formality.* The critical path runs through architectural decisions — the combinational bank query path of Section 4.8 is a design choice with a timing consequence.
- *That the difference between setup and hold violations is fundamental.* One costs frequency; the other costs the chip.
- *That the hardest bugs are the legal ones.* The defects documented in Section 10.2 all compiled, elaborated and synthesised without error. Indexing a vector with a bank index is valid Verilog that means something entirely different from what was intended.
- *That verification gaps must be stated.* Section 6.7 lists six of them. A design verified by waveform inspection against stimulus rather than against a device model is verified to a specific and limited degree, and saying so is what makes the rest of the report credible.

Industrial relevance. The specific artefact — a single-data-rate, read-only, single- transaction SDRAM controller — is not a product. Its architecture, however, is the architecture of every DRAM controller: address decode, bank state tracking, command sequencing, timing enforcement, refresh management. A DDR5 controller adds source- synchronous strobes, training, deep scheduling and an analogue PHY on top of this same skeleton. The flow traversed is likewise the flow used commercially, with open tools substituted for licensed ones at each stage — the stage names, the intermediate databases, the reports and the failure modes are the same.

Honest scope statement. This is a V1 implementation. It reads but does not write; it initialises the device but does not refresh it during operation; it handles one transaction at a time with no burst support and no queueing; its data path from the memory device is not yet closed; and its verification is directed and inspected rather than automated and self-checking. Chapter 11 sequences the work required to address each of these. Presenting those limitations explicitly is a deliberate choice: an engineering document whose scope statement is accurate is more useful, and more credible, than one whose claims must be discounted by the reader.

References

- [1] JEDEC Solid State Technology Association, *Synchronous Dynamic Random Access Memory (SDRAM) Specification*, JESD21-C, Section 3.11.5.
- [2] JEDEC Solid State Technology Association, *DDR4 SDRAM Standard*, JESD79-4, Arlington, VA, USA.
- [3] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*, 4th ed. Boston, MA, USA: Addison-Wesley, 2010.
- [4] J. M. Rabaey, A. Chandrakasan, and B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.
- [5] B. Jacob, S. W. Ng, and D. T. Wang, *Memory Systems: Cache, DRAM, Disk*. Burlington, MA, USA: Morgan Kaufmann, 2007.
- [6] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.
- [7] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, “Memory access scheduling,” in *Proc. 27th Annu. Int. Symp. Computer Architecture (ISCA)*, Vancouver, BC, Canada, 2000, pp. 128–138.
- [8] W. A. Wulf and S. A. McKee, “Hitting the memory wall: Implications of the obvious,” *ACM SIGARCH Computer Architecture News*, vol. 23, no. 1, pp. 20–24, Mar. 1995.
- [9] M. Shalan and T. Edwards, “Building OpenLANE: A 130nm OpenROAD-based tapeout-proven flow,” in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2020.
- [10] T. Ajayi et al., “OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain,” in *Proc. Government Microcircuit Applications and Critical Technology Conf. (GOMACTech)*, 2019.
- [11] C. Wolf, *Yosys Open SYnthesis Suite Manual*. [Online]. Available: <https://yosyshq.net/yosys/documentation.html>
- [12] Efabless Corporation, *OpenLane Documentation*. [Online]. Available: <https://openlane.readthedocs.io>
- [13] SkyWater Technology Foundry and Google LLC, *SkyWater SKY130 PDK Documentation*. [Online]. Available: <https://skywater-pdk.readthedocs.io>
- [14] R. N. Mayo, M. H. Arnold, W. S. Scott, D. Stark, and G. T. Hamachi, “1990 DECWRL/ Livermore Magic release,” Digital Equipment Corp. Western Research Lab., Tech. Rep. 90/7, 1990.
- [15] M. Köfferlein, *KLayout Documentation*. [Online]. Available: <https://www.klayout.de/doc.html>
- [16] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2001, 2001.
- [17] IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language, IEEE Std 1800-2017, 2018.
- [18] C. E. Cummings, “Coding and scripting techniques for FSM designs with synthesis-optimized, glitch-free outputs,” in *Proc. Synopsys Users Group (SNUG)*, Boston, MA, USA, 2000.
- [19] C. E. Cummings and D. Mills, “Synchronous resets? Asynchronous resets? I am so confused! How will I ever know which to use?,” in *Proc. Synopsys Users Group (SNUG)*, San Jose, CA, USA, 2002.
- [20] Micron Technology Inc., *Synchronous DRAM Datasheet and Verilog Simulation Model*, Boise, ID, USA.
- [21] ARM Limited, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022.
- [22] J. Lu et al., “ePlace: Electrostatics-based placement using Nesterov’s method,” in *Proc. 51st ACM/EDAC/IEEE Design Automation Conf. (DAC)*, 2014.

Populate access dates and specific document revisions when finalising. For a GitHub release, hyperlink the online references directly.

Appendix A — Module Hierarchy

```
sdrām_controller          (top level, structural only)
├── u_address_decoder      : address_decoder
│   in  : cpu_address[23:0]
│   out : bank[1:0], row[11:0], column[7:0]
├── u_sdrām_init           : sdrām_init
│   in  : clk, reset
│   out : init_done, precharge_all_cmd, refresh_cmd, load_mode_cmd
├── u_bank_manager         : bank_manager
│   in  : clk, reset, activate_cmd, precharge_cmd, precharge_all_cmd,
│         bank_cmd[1:0], row_cmd[11:0], query_bank[1:0]
│   out : bank_open, active_row[11:0]
├── u_command_generator    : command_generator
│   in  : clk, reset, init_done, read_request,
│         bank[1:0], requested_row[11:0], column[7:0],
│         bank_open, active_row[11:0]
│   out : activate_cmd, read_cmd, precharge_cmd,
│         bank_cmd[1:0], row_cmd[11:0], column_cmd[7:0],
│         read_data[31:0], data_valid, ready, req_ack, status_bank[1:0]
```

Figure A.1 placeholder — [Insert Figure A.1: Module hierarchy tree]

Appendix B — RTL File Descriptions

Table B.1 — RTL source file descriptions

File	Lines	Contains	Sequential?
<code>sdram_controller.v</code>	<n>	Top level: port declarations, internal wires, four instantiations	No
<code>address_decoder.v</code>	<n>	Combinational bit-field extraction	No
<code>bank_manager.v</code>	<n>	Per-bank open-row state array and query multiplexer	Yes
<code>sdram_init.v</code>	<n>	Power-up sequencer FSM and delay counter	Yes
<code>command_generator.v</code>	<n>	Main control FSM, address latches, timing counter	Yes
<code>sdram_controller_tb.v</code>	<n>	Testbench: clock, reset, directed stimulus	—

Appendix C — Repository Folder Structure

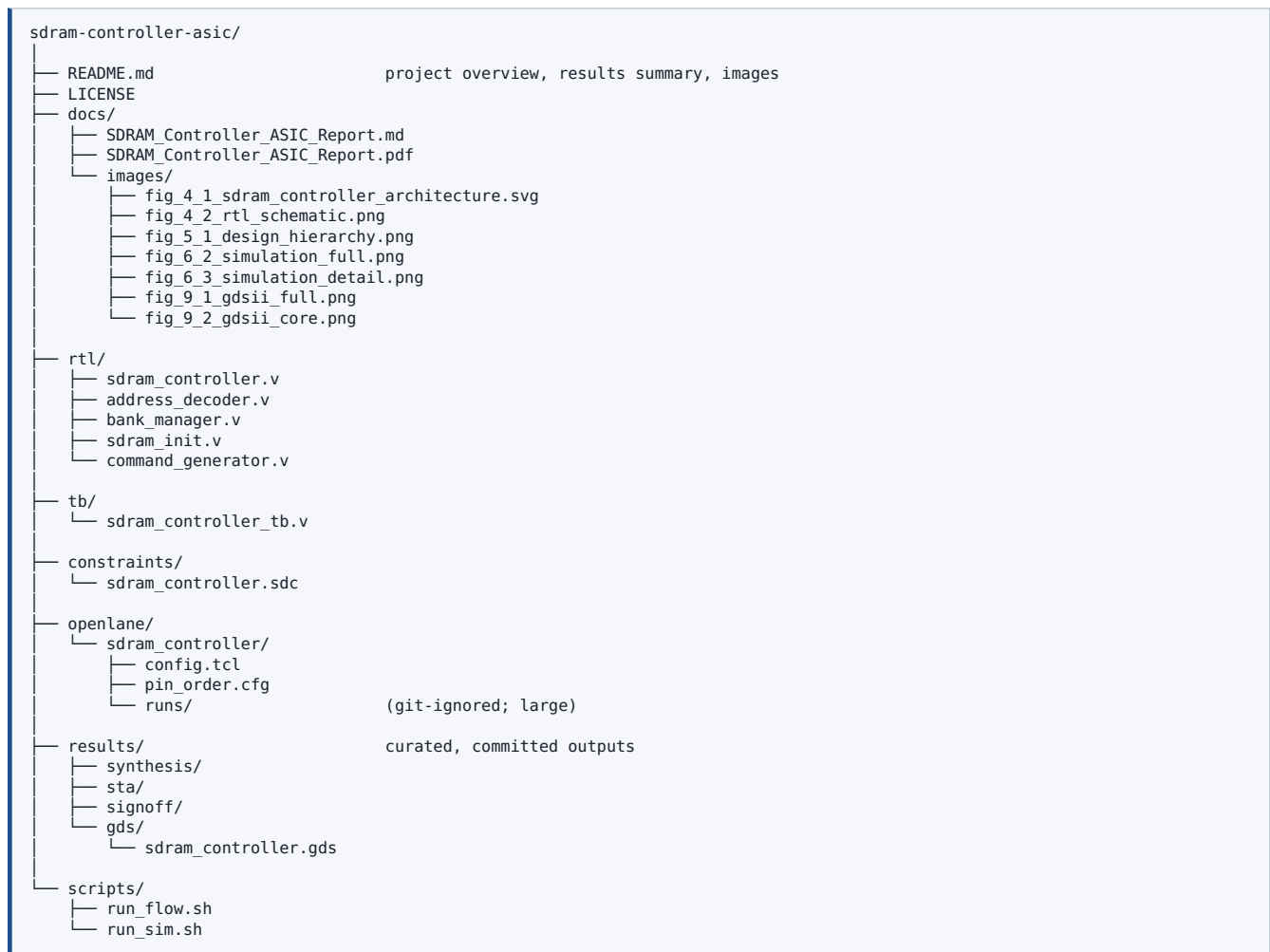


Figure C.1 placeholder — [Insert Figure C.1: Repository structure]

Note on repository hygiene. OpenLane run directories are large and contain many intermediate databases. Add `openlane/*/runs/` to `.gitignore` and commit only curated results — the final GDS, the key reports, and the figures used in this document. A repository that a reviewer can clone in seconds is materially more likely to be looked at than one that is several gigabytes.

Appendix D — ASIC Flow Screenshot Checklist

Capture each of the following from the OpenLane run to complete Chapter 9. Stage DEFs open directly in KLayout; the OpenROAD GUI gives better congestion and clock-tree views.

Figure	Content	Source
7.2	Synthesis statistics	reports/synthesis/1-synthesis.stat.rpt
7.3	Floorplan with core box and I/O pins	results/floorplan/*.def
7.4	PDN straps and rails	results/floorplan/*.def post-pdngen
7.5	Global placement density map	OpenROAD GUI heat map
7.6	Legalised detailed placement	results/placement/*.def
7.7	Clock tree	OpenROAD GUI clock tree view
7.8	Global routing congestion	OpenROAD GUI congestion map
7.9	Detailed routing, all layers	results/routing/*.def
7.10	Per-layer metal utilisation	routing reports
9.1	Final GDS, full die — captured	images/Screenshot_2026-07-28_224303.png
9.2	Final GDS, core detail — captured	images/Screenshot_2026-07-28_224352.png
9.3	Timing histogram	STA reports
9.5	DRC summary	reports/signoff/
9.6	LVS summary	reports/signoff/

Capture guidance. Use a consistent zoom and background across figures. Disable net-name label rendering for layout figures — labels obscure the geometry and reproduce poorly at print size. Export at a minimum of 1920 px wide. For layout views, a black background renders metal layers best on screen but a white background reproduces better in a printed PDF; pick one and apply it to every layout figure.

Appendix E — Simulation Screenshot Checklist

Figure	Content	Status
6.1	Testbench block diagram	To be drawn
6.2	Full-run waveform	Captured — Screenshot_2026-07-27_160236.png
6.3	ACTIVATE–READ–PRECHARGE detail	Captured — Screenshot_2026-07-27_161510.png
6.4	Bank manager internal signals	Captured — Screenshot_2026-07-25_110348.png
6.5	Initialization sequence waveform	To be captured
6.6	Row-hit versus row-conflict comparison	To be captured
6.7	Reset-during-transaction recovery	To be captured

The three uncaptured figures correspond to test cases TC-02, TC-09/TC-10 and TC-12 respectively, and capturing them would substantially strengthen Chapter 6 — TC-09 versus TC-10 in particular, because a side-by-side waveform showing ACTIVATE present in one case and absent in the other is the clearest possible demonstration that the bank manager is doing its job.

Appendix F — Reproduction Instructions

```
# 1. Clone
git clone https://github.com/<user>/sdram-controller-asic.git
cd sdram-controller-asic

# 2. Simulation (Icarus Verilog)
iverilog -o sim.out rtl/*.v tb/sdram_controller_tb.v
vvp sim.out
gtkwave dump.vcd

# 3. Physical implementation (OpenLane)
export PDK_ROOT=<path to pdk>
cd <openlane install>
make mount
# inside the container:
./flow.tcl -design <path>/openlane/sdram_controller

# 4. Results
# runs/<run_tag>/results/final/gds/sdram_controller.gds
# runs/<run_tag>/reports/
```

Requirement	Version used
Docker	<version>
OpenLane	<version>
SKY130 PDK	sky130A , commit <hash>
Icarus Verilog (optional)	<version>
Host OS	<OS and version>

Recording the exact PDK commit hash matters more than it appears to: the SKY130 PDK is actively maintained, and cell library or rule deck changes between commits can alter results. A run that cannot be reproduced is a claim, not a result.

End of document